Ziren: Processor Design Meets Verifiable Computation

A MIPS32r2 Intellectual-Property Core in Arithmetic Circuits

The ZKM Project*

September 8, 2026

Abstract

Ziren is a succinct argument system for the execution of MIPS32r2 programs, and, to our knowledge, the first such system for the MIPS instruction set; this paper describes its second generation. We present it as a processor intellectual-property core: a MIPS32r2 core whose functional units, register file, memory subsystem and interconnect are realised as arithmetic circuits over a 31-bit field, with ports, a configuration space and integration paths, which emits for every execution a proof small enough to publish and cheap enough to check inside another proof.

Every executed instruction is one row of an opcode-specific unit carrying its own instruction frame, so there is no central control table; registers and memory share one multiset argument with an elliptic-curve digest across shards; and all relations of a shard are discharged by one lookup argument and one zerocheck, with its tens of thousands of columns stacked into a single multilinear polynomial and proved by the jagged reduction over WHIR. Knowledge soundness is stated in the random-oracle model under the unique-decoding radius of Reed–Solomon codes, with no list-decoding conjecture, and under a hardness assumption for the cross-shard elliptic-curve digest, which we state explicitly and identify as the weakest link.

We verify the core as a processor is verified, against specification-derived vectors, an executable Lean 4 model of the instruction set, and determinism theorems extracted mechanically from the constraint system; the flow found and fixed a genuine under-determination at the syscall bus. Two efficiency metrics are reported on one host. Emulation runs at 69 MHz of guest execution through the compiled producer, an order of magnitude above the event-recording interpreter it replaces. Proving reaches 5.9 MHz on one NVIDIA RTX 5090, where an Ethereum block of 288 million MIPS cycles becomes a 603 KiB proof in 49 s, and 20 MHz on four GPUs for a larger block. Area in trace cells is reported per unit and throughput per accelerator, as gate count and frequency would be.

1 Introduction

A *succinct argument for machine execution* lets a prover convince a verifier that a program, run on a given instruction-set architecture, produced a given output, with a proof whose verification cost is essentially independent of the length of the run. Since 2022 such arguments have been built for RISC-V by a series of systems [Suc24, RIS24b, Pol25, Mat25, LZZ+24, Axi24, AST24], generically called zero-knowledge virtual machines (zkVMs) after the proof systems they use. They are general-purpose: any program the guest toolchain compiles can be proved, and the statements people prove are dominated by *hashing*—Merkle inclusion and update, signature and certificate chains, content addressing, firmware and log attestation—because a hash is the standard way to commit to data that a verifier does not hold. The workload on which such systems are publicly compared, proving an Ethereum block of several hundred million instructions in tens of seconds on one GPU [Eth26b], is itself hash-dominated for that reason, and we use it as a benchmark rather than as the application. The proofs in this line of work, including ours, are *succinct* and *knowledge sound* but not hiding: no blinding is applied to the committed execution trace. Section 1.1 fixes the statement precisely.

MIPS32r2 is a different instruction set with a different constituency. It is mature, stable and simple: fixed 32-bit encodings, a handful of formats, three decades of production use and complete toolchains (GCC, LLVM,

*Project ZKM. Source, artifacts and measurements: https://github.com/ProjectZKM/Ziren.

the Linux kernel, musl and a Rust target). It is the instruction set executed by the fault-proof infrastructure of Ethereum rollups, Optimism's Cannon and Kona [Opt23, OP 24], so a MIPS argument system lets one program image be both fault-proved and validity-proved. And it is deployed at a scale few architectures match: more than 8.5 billion MIPS devices have shipped [MIP26], in the router and gateway families of MediaTek and Qualcomm Atheros that are the two largest OpenWrt targets [Med26, Ope26], in Microchip's PIC32M microcontrollers [Mic26], and in set-top-box and broadband SoCs. Since its owner moved its own roadmap to RISC-V in 2021 [Tur21, Har25] the architecture no longer changes, which for an argument system is a benefit: the specification to implement is fixed. We therefore ask:

Can the MIPS32r2 instruction set be given a succinct argument system whose prover is competitive with the RISC-V systems, whose soundness is stated under the weakest standard assumptions, and whose constraint system is itself verified?

The first generation of Ziren, zkMIPS [ZKM24], was to our knowledge the first zkVM for MIPS: a single CPU-table design over the Goldilocks field, an order of magnitude slower than the RISC-V systems of its time. Cannon [Opt23] executes MIPS inside a fault-proof game but adjudicates one instruction at a time on-chain rather than proving an execution succinctly. This paper describes the second generation of Ziren [The25], and its position is that the right way to describe and evaluate such a system is as a *processor IP core*: functional units, buses, a register file and a memory subsystem, realised not in logic gates but as tables of field elements constrained by low-degree polynomial identities, with an area metric (trace cells), a throughput metric (proved guest cycles per second on a given accelerator), and a verification flow (Section 4). Our first result is the argument system itself.

Theorem 1.1 (Argument system, informal)**.** *Let* Emulate *be the MIPS32r2 execution relation of Section 3. There is a non-interactive argument of knowledge for* Emulate *over the 31-bit field* KoalaBear*, built from one lookup argument and one zerocheck per shard of a few million cycles, a jagged multilinear commitment opened through the* WHIR *proximity test, and a recursion tree, with the following properties. Completeness is unconditional. Knowledge soundness holds in the random-oracle model for the* Poseidon2*-based Fiat–Shamir transcript, under the unique-decoding radius of Reed–Solomon codes (no list-decoding conjecture) and a stated hardness assumption for the cross-shard elliptic-curve digest (Assumption 5.4), with an error bounded by the sum of the errors of the transcript's components (Theorem 7.1). On one NVIDIA RTX 5090, an Ethereum block of 288 million MIPS cycles is proved to a 603 KiB compressed proof in 74 s in the deployed configuration and in 49 s after the changes reported here; four GPUs in one host reach 20 MHz of proved execution.*

The construction is simple to describe. Every executed instruction is one row of the unit that implements its opcode, and the row carries an *instruction frame*: it fetches its instruction from a committed program ROM on a *program bus*, performs its register reads and its register write at fixed sub-cycle positions on a *memory bus*, and receives the machine state $(\mathsf{shard}, \mathsf{clk}, \mathsf{pc}, \mathsf{next\_pc})$ of its cycle and sends that of the next cycle on a *state bus*. There is no central control table. Registers and memory share one offline multiset argument; its cross-shard part is a running elliptic-curve digest. Every bus is a multiset relation, and all of them are discharged by one logarithmic-derivative lookup argument per shard, while all polynomial identities are discharged by one zerocheck. The tens of thousands of columns of a shard, of widely differing heights, are concatenated into one dense vector, committed as one multilinear polynomial, and the column claims are reduced to it by the *jagged* sumcheck of Hemo et al. [HJR+25], the reduction for tables of differing heights, whose verifier depends only on the logarithm of the trace area. That polynomial is opened with WHIR [ACFY24b], an interactive oracle proof of proximity (IOPP) for Reed–Solomon codes, whose three committed rounds fold several variables at a time while the code rate falls from round to round.

Concrete efficiency. The argument is not merely asymptotic. Table 1 places the deployed system next to the single-GPU provers of the public block-proving dashboard [Eth26b] on the day of measurement; the systems differ in field, security regime and proof size as much as in time, so we report the comparison as context (Section 10.6) and draw no ranking from it. The rest of the paper's measurements are same-hardware and same-input: a per-unit area census with a *(row, interaction)* cost model, the effect of three arithmetisation changes and one configuration change (-9.4% together, of which the configuration change is the largest single contribution), the effect of the lookup circuit's layer order (-28%), and scaling from one to four GPUs (Section 10).

Table 1: Single-GPU block-proving provers on the public dashboard [Eth26b], 23-hour medians on 5 September 2026, with the field and the proof size. The last two rows are Ziren's deployed configuration and the configuration after the change of §10.5, both measured on the block of §10.1. They are not comparable with the row above them: the dashboard median is over other blocks, this paper's block is 288 rather than 379 million cycles (§10.1), and the proof-size difference is the commitment schedule of §7.3 rather than any property of the block.

System	ISA	Field	time (s)	proof
Airbender [Mat25]	RV32IM	Mersenne-31	42	—
ZisK [Pol25]	RV64IMA	Goldilocks	73	—
Ceno [LZZ+24] (RTX 4090)	RV32IM	Goldilocks	74	—
cysic [Cys25] (RTX 4090)	RV32IM	—	92	—
Ziren, dashboard median at the time	MIPS32r2	KoalaBear	107	1.0 MB
Ziren, deployed configuration, this paper's block	MIPS32r2	KoalaBear	74	603 KiB
Ziren, after the lookup-layer change (§10.5)	MIPS32r2	KoalaBear	49	603 KiB

Beyond proving: verifying the processor. Soundness of the argument says that an accepting proof implies a satisfying trace. Whether that trace is an execution of the advertised machine is a separate question, and it is the question a processor verification team asks before tape-out: does the emulator implement the ISA, and does each unit's constraint system admit exactly one behaviour per input? Our second result is a verification flow that treats both with the tools of hardware verification, and whose central object is extracted from the constraint system itself.

Theorem 1.2 (Unit determinism, informal)**.** *For every unit U of the core machine there is a mechanically extracted Lean 4 theorem stating that two rows of U that satisfy its constraints and agree on their bus inputs agree on their bus outputs. Of the 106 such theorems, 17 are closed by a tactic that lifts the statement to bounded integer arithmetic and replays the propagation loop of Picus [PCW+23] inside the proof assistant; the remainder are open and classified in Figure 7. Together with the bus arguments, unit determinism implies that an accepting shard proof determines, up to permutation, every row reachable from the entry state along the state and memory buses (Theorem 5.5); that this exhausts the real rows is a property of the unit set, which we argue but do not prove.*

The flow found real defects: the syscall unit's argument half-words were not determined by the reduced argument it received (because 2^{32} exceeds the field modulus), and its Linux selector had no input pinning it. Both were fixed by carrying the exact half-words and the selector on the syscall bus, after which the unit's theorem closes (Section 9.3).

1.1 The statement being proved

Let $\mathsf{Emulate}$ be the deterministic MIPS32r2 execution relation of Section 3, the relation the emulator realises: given a program image P (code and initial data), an input stream in and a host-supplied *hint* stream hint, it produces an output stream out and an exit status e, or diverges. Ziren proves the relation

$$\mathcal{R} \;=\; \big\{\big((\mathsf{vk},\ h,\ e),\ (P, \mathsf{in}, \mathsf{hint})\big) \;:\; \mathsf{vk} = \mathsf{Setup}(P) \ \wedge\ \mathsf{Emulate}(P, \mathsf{in}, \mathsf{hint}) = (\mathsf{out}, e) \ \wedge\ h = H(\mathsf{out})\big\},$$

where Setup maps a program image to its verifying key, a Merkle commitment to the program ROM and the preprocessed tables of Section 5, and H is SHA-256 over the committed output. The program image, the input stream and the hint stream are the witness: the verifying key pins the image, and the verifier learns of the two streams only their effect on the output. The proof system is *complete* (an honest execution always yields an accepting proof) and *knowledge sound* with the error bound of Theorem 7.1: from a prover that makes the verifier accept with probability noticeably above that bound, one can extract $(\mathsf{in}, \mathsf{hint})$ for which $\mathsf{Emulate}$ produces the claimed output. No zero-knowledge property is claimed.

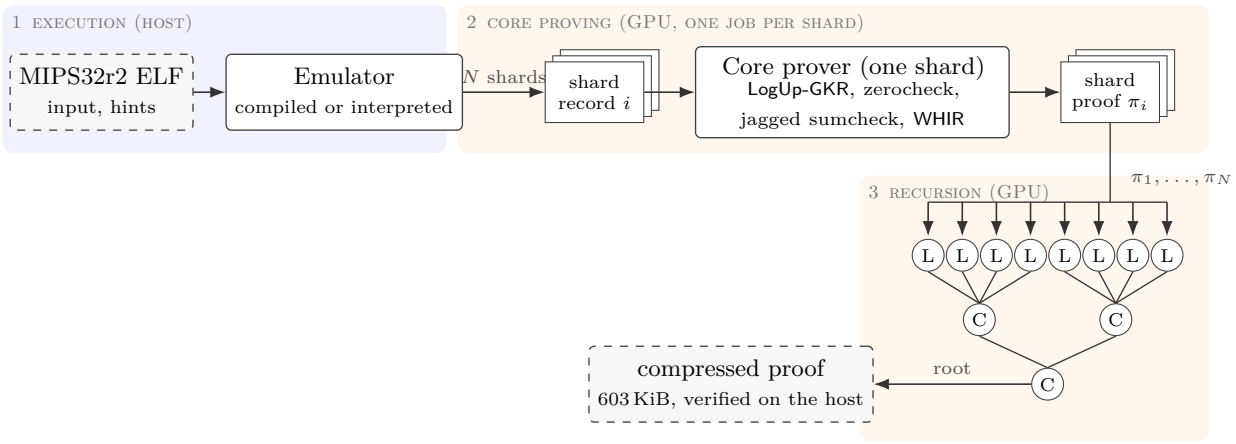


Figure 1: The proving pipeline. The emulator cuts the run into N shards of a few million cycles and emits one record per shard; each record is proved independently as one Jagged-WHIR shard proof π_i; leaf programs (L) verify one or two shard proofs each and a range tree of compose programs (C, arity at most four) merges them; the root of the tree is the *compressed proof*, the object measured throughout this paper.

1.2 Techniques

This section gives an overview of the design and of the arguments behind the two theorems; the sections that follow give the definitions and proofs. Figure 1 shows the proving pipeline and Figure 2 the shard proof as a protocol.

The four ideas that carry the design are developed in the sections named. The *instruction frame* (§5.2) removes the central control table that the first generation and SP1 before its hypercube release both had: each opcode unit fetches its own instruction, performs its own accesses at fixed sub-cycle positions and passes the machine state on a bus, so a cycle costs one row of one unit rather than a row of a wide table plus a row of a narrow one. *One lookup and one zerocheck* per shard (§5.8, §5.9) discharge every bus and every polynomial identity, batched across units and sharing one opening per column. *Stacking and the jagged reduction* (§6) commit a shard's tens of thousands of columns, whose heights span five orders of magnitude, as a single multilinear polynomial whose verifier depends only on the logarithm of the trace area, so a unit can be widened or removed without changing the program that verifies the shard. And *unit determinism* (§9.2) is the property that makes an accepting proof pin down the execution rather than merely some satisfying assignment: two satisfying rows of a unit agreeing on their bus inputs agree on their bus outputs, which the bus lemmas then propagate along the shard.

1.3 Scope and limitations

The proofs are not zero-knowledge. The soundness statement rests on a computational assumption for the cross-shard digest in addition to the information-theoretic bounds of the proximity test, and the numerical level of a configuration is reported through the artifact's accounting rather than restated as a single figure. The determinism theorems cover the core machine and not the recursion machine, a substantial fraction of them remain open under the automation's budget, and Theorem 5.5 is stated, not machine-checked. The evaluation uses one workload class (Ethereum blocks) on one GPU model. The verifying-key allowlist that binds the recursion to the advertised machine is collected from executions for the core programs and therefore covers the block guest rather than an arbitrary program. Timing closure and power have no counterpart in this setting, and area is measured in trace cells rather than gates.

1.4 Open problems

The questions the design leaves open are, in the terms of Section 4, about the block's area, its verification collateral and its portability.

The shard proof.
Input: a shard record (per-unit event lists, touched-address ledger, public values) and the verifying key (Merkle commitment to the program ROM and the preprocessed tables).

1. **Trace and commitment.** Each unit's events become rows of its table; row counts are rounded to multiples of 32. The per-unit dimensions are absorbed into the transcript. All columns are concatenated into one dense vector $\mathbf{q}$ of length A (the trace area), cut into *stripes*, blocks of consecutive entries of a fixed height, then Reed–Solomon encoded and Merkle-committed once (Section 6).
2. **Lookup argument.** The verifier samples α, β. Every bus interaction of every row contributes a fraction $m/(\alpha - \mathsf{bus} - \sum_{j \geq 1} \beta^j f_j)$; a layered GKR circuit sums sends and receives and the prover shows the two sums agree, layer by layer by sumcheck (Section 5.8). The final layer's claims are evaluations of unit columns at a random point.
3. **Zerocheck.** The verifier samples the zerocheck challenges. The prover shows by one sumcheck that the random combination of all units' constraints vanishes on their real rows (Section 5.9), seeded from the lookup openings so that each column is opened once.
4. **Jagged reduction.** The $\approx 3.6 \cdot 10^4$ column claims are batched and reduced by one degree-two sumcheck to a single claim $\widetilde{\mathbf{q}}(\boldsymbol{z}^*) = v$ on the dense polynomial; the verifier evaluates the jagged indicator through a width-four branching program in $O(\log A)$ operations (Section 6.2).
5. **Proximity test.** The claim on $\widetilde{\mathbf{q}}$ is proved with WHIR over three committed rounds, each folding several variables, re-encoding at a lower rate, drawing out-of-domain samples and answering in-domain queries under proof-of-work grinding (Section 7); the final polynomial is sent in the clear.

Output: a shard proof of roughly a megabyte whose public values carry the entry and exit state, the address cursors and the digest of its cross-shard interactions, a point on a curve over a degree-seven (*septic*) extension of the field; a block's shard proofs total about 112 MB before recursion (§10.4).

Figure 2: The shard proof as a protocol.

Can the trace area be halved? Area is what the prover commits and opens, so halving it is this design's counterpart of a shrink (§4), and two items dominate. The lookup circuit's cost is the number of (row, interaction) pairs, and the memory-instruction units carry 21 byte-table interactions per row; merging byte checks across operands, or omitting them where the frame already guarantees byte shape, removes many pairs without touching a column count. Separately, every word touched in a shard costs two rows of the cross-shard digest unit, a quarter of the block's area, which a digest of the per-shard *delta* or a larger shard would reduce—the latter bounded by the memory of the lookup circuit's first layer.

Can the collateral cover the whole block? What is verified today is the core machine's units. The recursion machine, which verifies proofs in-circuit and is where the composition of Theorem 7.1 takes its hypotheses, is outside the extraction entirely; and determinism is weaker than correctness, since it says a unit has one behaviour rather than the right one. Closing either gap—extracting the recursion units, or proving units against the executable model rather than only against each other—would change what an integrator may assume without re-verifying.

How portable is the realisation? Throughput belongs to the core and the accelerator together (§4), and every figure here comes from one GPU model. Which parts of the cost model survive a different memory hierarchy, and whether this constraint system is the right design for an accelerator with a different arithmetic-to-bandwidth ratio, is untested.

1.5 Related work

Table 2 places the systems discussed along the axes of this introduction: guest instruction set, arithmetisation, lookup argument, and commitment with its proximity test.

Table 2: Design axes of the block-proving zkVMs discussed in the text, from their public descriptions. AIR is an algebraic intermediate representation, PCS a polynomial commitment scheme and IOPP an interactive oracle proof of proximity. The last column names the regime under which each project states its level; regimes and conjectures differ and levels are not comparable without the accounting of Section 7.3.

System	ISA	Field	Instruction tables	Lookup	Commitment / IOPP	Regime of stated level
Ziren	MIPS32r2	KoalaBear (2^{31})	unit per opcode, frames	LogUp-GKR	Jagged over WHIR	unique decoding, union bound
SP1 Hypercube [Suc25a]	RV32IM	KoalaBear	chip per opcode	LogUp-GKR	Jagged over BaseFold	project target
RISC Zero [RIS24b]	RV32IM	BabyBear	CPU circuit, continuations	permutation / lookup	DEEP-ALI + FRI	project calculator
ZisK [Pol25]	RV64IMA	Goldilocks	micro-ops, PIL2	LogUp	FRI (STARK)	per release
Airbender [Mat25]	RV32IM	Mersenne-31	degree-2 AIR	LogUp	DEEP-FRI	project target
Ceno [LZZ+24]	RV32IM	Goldilocks	basic-block circuits	GKR-based	multilinear PCS	—
OpenVM [Axi24]	RV32IM + ext.	BabyBear	modular emulator chips	LogUp	FRI (Plonky3)	project target
Jolt [AST24]	RV32IM	256-bit / small	Lasso opcode tables	Lasso	multilinear PCS	—
Cairo [Sta21]	Cairo (own)	251-bit	CPU AIR, builtins	permutation	FRI	—

Chip-decomposed machines. SP1 [Suc24] established the template Ziren follows—a unit per opcode, LogUp-GKR, recursive aggregation—and SP1 Hypercube [Suc25a] added the jagged commitment and the shard-level lookup and zerocheck shape; its just-in-time executor [Suc25b] is the model for ours. Ziren inherits that template and the septic cross-shard digest, and differs in the guest instruction set, in the register argument (clock-zero shadow reads rather than a shard field on every register access), in the proximity test, and in reporting its accounting in the unique-decoding regime under a union bound rather than against a target. OpenVM [Axi24] likewise has no central control table; the instruction frame is that idea specialised to MIPS. Valida [Lit24] shares the ancestry but co-designed a proof-oriented instruction set, as Cairo [Sta21] did earlier—a design point closed to us, since the value of MIPS here is precisely that it is a hardware architecture with existing toolchains and an existing fault-proof ecosystem. RISC Zero [RIS24b] instead proves one CPU circuit with continuations, paying every instruction's cost on every cycle.

Where the prover's work is put. Jolt [AST24] evaluates instructions by indexed lookups into structured tables [STW24], moving the dominant cost into the lookup argument and its multiplicities; Ziren uses lookups for byte-granular facts and for buses, and evaluates instructions by polynomial constraints. Ceno [LZZ+24] proves per-basic-block circuits with GKR at the instruction level, where ours is confined to the lookup argument. ZisK [Pol25] and Airbender [Mat25] are the other single-GPU provers on the public dashboard [Eth26b]; Nexus [Nex24] takes a folding route we do not measure against.

Fields, proximity and commitment. The field, the Poseidon2 permutation [GKS23], the constraint interfaces and the transforms come from Plonky3 [Pol24]. Circle STARKs [HLP24] reach smooth domains over Mersenne-31 through the circle group; KoalaBear has two-adicity 24 and uses ordinary Reed–Solomon domains, at the price of the word splitting of Section 2, which Binius [DP23, DP24] removes by working over binary towers. The proximity line runs from FRI [BSBHR18] through DEEP-FRI [BSGKS20] and the proximity-gap bounds [BSCI+20] to BaseFold [ZCF23], STIR [ACFY24a] and WHIR [ACFY24b]; Ligero [AHIV17] and Brakedown [GLS+21] are the linear-time alternatives we do not use. The jagged commitment is that of [HJR+25], with the branching-program evaluation of Holmgren and Rothblum [HR18]; our contribution at this layer is the composition of jagged with WHIR and its measured schedule. The lookup is Haböck's logarithmic derivative [Hab22] in its GKR realisation [PH23], used as published—Plookup [GW20] and cq [EFG22] are univariate alternatives that do not transfer to a multilinear, many-bus workload—and the memory argument descends from [BEG+94].

MIPS, embedded verification, and circuit verification. Cannon [Opt23] is a single-step MIPS interpreter for interactive fault proofs, and its suite is one of our conformance inputs; Kona [OP 24] is the fault-proof program of the same ecosystem. zkMIPS [ZKM24] is the first-generation prover this one replaces. Proofs reach constrained devices today as authentication and attestation protocols [CJSC23, SD21, DDGM23, EH24], where the statement is small and about the device; a zkVM for the device's own instruction set inverts that, making the execution of its software the statement and the constrained party the verifier. Picus [PCW+23] introduced the determinism formulation of under-constrained circuits and an SMT solver for it; Section 9.2 states the same property and discharges it in Lean 4, because we could not run the public solver at the unit sizes of Table 3. The accounting follows the round-by-round methodology of [ACFY24b] and is computed

with `soundcalc` [Eth26a]; RISC Zero's calculator [RIS24a] and the ethSTARK documentation [Sta23] are the comparable published treatments.

1.6 Organisation

Section 2 fixes notation and defines units, buses and determinism. Section 3 gives the instruction set and execution model, Section 4 what kind of object the core is and what may therefore be measured about it, Section 5 the arithmetisation with the bus lemmas and the uniqueness theorem, Section 6 the commitment, Section 7 the proximity test and the soundness composition, and Section 8 the recursion. Section 9 describes the verification flow and its findings, and Section 10 the concrete efficiency. Appendix A recalls the protocols as implemented, and Appendix B gives the outward-facing description of the core: its ports, its configuration parameters, the ways it is integrated and the collateral delivered with it.

2 Preliminaries

2.1 Field, polynomials and codes

Field. All traces live over the prime field $\mathbb{F} = \mathsf{KoalaBear}$, $p = 2^{31} - 2^{24} + 1$. Its multiplicative group has two-adicity 24, which bounds the Reed–Solomon domains available to the proximity test: the rate schedule of Section 7 is chosen so that the largest domain stays within that limit. Random challenges and all sumcheck arithmetic use the degree-four binomial extension $\mathbb{F}_{p^4} = \mathbb{F}[X]/(X^4 - 3)$; a sumcheck round polynomial of degree d therefore has soundness error $d/|\mathbb{F}_{p^4}| \approx d \cdot 2^{-124}$ per round. Note that $2^{32} > p$: a 32-bit machine word cannot be represented by a single field element, and every word in the arithmetisation is a vector of four field elements holding its little-endian bytes.

Multilinear extensions and sumcheck. For $f : \{0,1\}^n \to \mathbb{F}$ the multilinear extension $\widetilde{f} \in \mathbb{F}[X_1, \ldots, X_n]$ is the unique multilinear polynomial agreeing with f on the hypercube, $\widetilde{f}(\boldsymbol{z}) = \sum_{\boldsymbol{b}} f(\boldsymbol{b})\, \mathsf{eq}(\boldsymbol{b}, \boldsymbol{z})$ with $\mathsf{eq}(\boldsymbol{b}, \boldsymbol{z}) = \prod_i (b_i z_i + (1 - b_i)(1 - z_i))$. The sumcheck protocol [LFKN92] reduces a claim $\sum_{\boldsymbol{b} \in \{0,1\}^n} g(\boldsymbol{b}) = \sigma$ about a low-degree g to a claim $g(\boldsymbol{r}) = \sigma'$ at a random $\boldsymbol{r} \in \mathbb{F}_{p^4}^n$ in n rounds, each sending a univariate polynomial of degree $\deg g$. We write a claim about a multilinear $\widetilde{f}$ as the pair $(\boldsymbol{z}, v)$ meaning $\widetilde{f}(\boldsymbol{z}) = v$.

Reed–Solomon and constrained Reed–Solomon codes. For a smooth domain $L \subset \mathbb{F}^*$ (a coset of a subgroup of order 2^ℓ) the code $\mathsf{RS}[\mathbb{F}, L, m]$ is the set of functions $L \to \mathbb{F}$ that agree with a polynomial of degree $< 2^m$; its rate is $\rho = 2^m/|L|$. Identifying the coefficient vector of such a polynomial with a function on $\{0,1\}^m$, a codeword is also the evaluation of an m-variate multilinear $\widehat{f}$ at the points $(x, x^2, x^4, \ldots, x^{2^{m-1}})$, $x \in L$. Following [ACFY24b], the *constrained* code $\mathsf{CRS}[\mathbb{F}, L, m, \widehat{w}, \sigma]$ is the subset of $\mathsf{RS}[\mathbb{F}, L, m]$ whose multilinear satisfies $\sum_{\boldsymbol{b} \in \{0,1\}^m} \widehat{w}(\widehat{f}(\boldsymbol{b}), \boldsymbol{b}) = \sigma$ for a weight polynomial $\widehat{w}$; the weight $\widehat{w}(Z, \boldsymbol{X}) = Z \cdot \mathsf{eq}(\boldsymbol{X}, \boldsymbol{z})$ expresses the evaluation claim $\widehat{f}(\boldsymbol{z}) = \sigma$. A *fold* of a codeword by a challenge α maps $f : L \to \mathbb{F}$ to $\mathsf{Fold}(f, \alpha)(x^2) = \frac{f(x) + f(-x)}{2} + \alpha \frac{f(x) - f(-x)}{2x}$, which is a codeword of $\mathsf{RS}[\mathbb{F}, L^{(2)}, m - 1]$ whenever f is a codeword, and equals the partial evaluation of $\widehat{f}$ at $X_1 = \alpha$.

Hashing and transcripts. Merkle commitments and the Fiat–Shamir transcript [FS87] use the Poseidon2 permutation [GKS23] over $\mathbb{F}$ at width 16 with a 248-bit digest (eight field elements). Its S-box has degree three, so the round schedule is the one tabulated for a 31-bit prime at that width and degree, eight full and twenty partial rounds; a lower S-box degree needs *more* partial rounds for the same margin, and the count for degree seven is thirteen. The same permutation is used by the transcript, the commitment trees and the recursion machine, so one schedule fixes all three—and, as Table 9 records, the configuration measured in Section 10 carried the degree-seven count. Proof-of-work grinding of b bits means the prover searches for a nonce whose absorption yields b leading zero bits in the squeezed challenge; it multiplies the cost of the corresponding cheating strategy by 2^b and appears additively in the security accounting.

2.2 Units, buses and determinism

Definition 2.1 (Unit)**.** A *unit* (a *chip* in the implementation and in the literature this design descends from) is a table over $\mathbb{F}$ of a fixed width w together with a set of polynomial constraints of degree at most three in the w columns of one row, a distinguished boolean column $\mathsf{is\_real}$ gating every constraint, and a set of *interactions*. A row is *satisfying* if it satisfies every constraint.

Definition 2.2 (Interaction and bus)**.** An interaction of a unit is a triple $(\mathsf{bus}, \boldsymbol{f}, m)$ where bus is a name, $\boldsymbol{f}$ is a tuple of linear expressions in the row's columns and m is a linear expression in the columns (the multiplicity), marked as a *send* or a *receive*. Multiplicities are non-negative integers bounded well below p: on the sending side of the arithmetisation of Section 5 they are zero, one, or a value range-checked against the byte table, while the preprocessed `Program` and `Byte` tables receive with the number of times a row was used in the shard, which the shard's cycle budget bounds by 2^{25}. In either case a sum over the rows of a shard stays far below p and is read as an integer. For a set of tables, the *multiset of a bus* is $\{(\boldsymbol{f}(r))\}$ with multiplicity $\sum m(r)$ over the sends of all rows r of all units, and likewise for the receives. A bus *balances* if its send and receive multisets are equal; because the multiplicities stay small, the fraction identity of Section 5.8 over $\mathbb{F}_{p^4}$ is equivalent to this multiset equality.

Lookups as fractions. A lookup argument proves that two multisets of field-element tuples agree. In the logarithmic-derivative form of Haböck and its GKR realisation [Hab22, PH23]—written LogUp-GKR throughout—each tuple $\boldsymbol{f}$ on bus bus with multiplicity m contributes a fraction $m/(\alpha - \mathsf{bus} - \sum_{j\geq 1} \beta^j f_j)$ for random $\alpha, \beta \in \mathbb{F}_{p^4}$, senders and receivers must sum to the same value, and the sum is computed by a layered GKR circuit [GKR08] whose layers are proved by sumcheck. Section 5.8 proves all buses of a shard by one such argument.

Zerocheck. A *zerocheck* proves that a polynomial vanishes on a set of points: to show $g(\boldsymbol{b}) = 0$ for every $\boldsymbol{b}$ in a subcube, the verifier samples $\boldsymbol{r}$ and the prover runs a sumcheck on $\sum_{\boldsymbol{b}} \mathsf{eq}(\boldsymbol{b}, \boldsymbol{r})\, g(\boldsymbol{b})$, which is zero for every $\boldsymbol{r}$ if and only if g vanishes on the subcube, up to the sumcheck's error. Section 5.9 proves all polynomial constraints of a shard by one such check.

Definition 2.3 (Shard relation)**.** A *shard* is a set of tables, one per unit, together with public values pv (the entry and exit state, the address cursors of the global memory tables and the septic digest, Section 3). It is *valid* if every real row is satisfying and every bus balances, where the public-values table contributes the entry state as a send and the exit state as a receive on the state bus, and the program bus is received by the preprocessed program table with the recorded multiplicities.

Definition 2.4 (Inputs, outputs and determinism of a unit)**.** For a unit U, the *inputs* $I_U(r)$ of a row r are the values the row consumes from elsewhere: the tuples of its receives, the program tuple it fetches, and the previous-value part of its memory accesses. The *outputs* $O_U(r)$ are the values it produces for elsewhere: the tuples of its sends other than the fetch, and the new-value part of its memory accesses. Fetch and previous value are sends in the sense of Definition 2.2—the preprocessed table and the earlier access are their sources—but they are inputs of the row, and the direction of a bus interaction is therefore not by itself the direction of the data. U is *deterministic* if for all satisfying real rows r, r',

$$I_U(r) = I_U(r') \;\Rightarrow\; O_U(r) = O_U(r').$$

This is the notion of Picus [PCW+23] adapted to the interaction language: whatever a row consumes from another table is an input, whatever it produces for another table is an output.

Cost metrics. Throughout, the *trace area* of a shard or a block is the sum over units of rows × columns of the committed main trace, in cells; the *(row, interaction) count* is the number of bus interactions summed over all rows, which is the number of leaves of the lookup circuit; and *throughput* is the number of proved guest cycles per second of wall-clock time. Section 10 relates the three to measured prover time.

3 The instruction set and execution model

3.1 MIPS32r2 as a guest

The guest machine is a 32-bit MIPS32r2 core with 32 general-purpose registers (register 0 hard-wired to zero), the `HI`/`LO` pair written by multiplication and division, and a little-endian byte-addressed memory. Three features of the ISA shape the arithmetisation more than anything else.

Branch-delay slots. The instruction that follows a branch or jump is executed before control transfers. The machine state therefore carries a program counter *pair*: the address of the current instruction and the address of the one that follows it. A sequential instruction advances the pair as $(\mathsf{pc}, \mathsf{next\_pc}) \mapsto (\mathsf{next\_pc}, \mathsf{next\_pc}{+}4)$; a branch instruction sets $\mathsf{next\_next\_pc}$ to the target or to $\mathsf{next\_pc} + 4$. Consequently the successor of a sequential instruction is *data*, not $\mathsf{pc} + 4$: when the instruction sits in a delay slot its successor is the branch target. Every instruction row carries $\mathsf{next\_pc}$ as a witnessed column for this reason.

Unaligned and narrow accesses. `LB`, `LBU`, `LH`, `LHU`, `SB`, `SH` address individual bytes and half-words of a word, and `LWL`/`LWR`/`SWL`/`SWR` merge the left or right part of an unaligned word with a register. Memory is modelled at word granularity, so these instructions select and merge bytes of a word in-circuit; they get units of their own (Section 5.1).

Bit manipulation. `EXT`, `INS`, `WSBH`, `SEB`/`SEH`, `ROTR`, `CLO`/`CLZ` and the conditional moves are common in hashing and serialisation code; they are supported natively rather than lowered.

3.2 Syscalls and precompiles

Guest programs reach the host through `SYSCALL`. Besides input/output the syscall table exposes *precompiles*: SHA-256, Keccak-f[1600], the Poseidon2 permutation, curve arithmetic on secp256k1, secp256r1, BN254, BLS12-381 and Ed25519, field arithmetic for BN254 and BLS12-381, 256-bit modular and 256×2048-bit multiplication, and a garbled-circuit gate evaluator. A precompile reads and writes guest memory directly; each invocation becomes one or more rows of a dedicated unit whose memory accesses are timestamped at the invoking cycle.

Hash functions. Hashing is the dominant primitive of the workloads this class of machine is asked to prove, and it is the reason the accelerator interface exists: a general-purpose guest computing SHA-256 or Keccak in MIPS instructions pays tens of instructions per round, each of them a row of an instruction unit, whereas an accelerator computes a permutation in a handful of rows of a wide unit. Three hash families are accelerated. SHA-256 is split into its message-schedule extension and its compression rounds, each a unit driven by a control unit that sequences the invocation's memory accesses. Keccak-f[1600] is a sponge unit with its own control unit, and at 2 637 columns it is one of the two widest units of the machine (Table 3). Poseidon2 is accelerated for guests that verify proofs of this system inside the guest, and the same permutation is used by the machine itself for Merkle commitments and the Fiat–Shamir transcript (Section 2) and appears again as a unit of the recursion machine (Section 8); it is therefore the one primitive that occurs in all three roles—guest accelerator, commitment hash and recursion unit. Hint data written by the host into previously untouched memory becomes part of the memory's initial state (Section 5.3), so the prover cannot inject values that escape the range discipline of the argument.

3.3 Clock, shards and the execution record

Execution is divided into *shards*. Within a shard a clock clk counts one tick per instruction; a syscall additionally advances it by a bounded number of ticks so that the memory accesses of a precompile fit strictly between the calling instruction and its successor. Each instruction's register and memory accesses occupy fixed *sub-cycle positions* $1, \dots, 4$ of its tick (Section 5.2), so a $(\mathsf{shard}, \mathsf{clk}, \mathsf{pos})$ triple identifies every access uniquely, and the memory argument timestamps an access by $4\,\mathsf{clk} + \mathsf{pos}$. The clock is bounded by 2^{25} and is witnessed as a 16-bit limb and a 9-bit limb; the shard closes when $\mathsf{clk} + \mathsf{max\_syscall\_cycles}$ would reach the

configured shard size, a few million cycles (Section 10 measures four values), or earlier if any unit's row count would exceed its height budget.

At a shard boundary the emulator emits, for every memory word touched in the shard, the value and timestamp of its first and last access; these are the leaves of the cross-shard memory argument. The record of a shard—per-unit event lists, the touched-address ledger and the shard's public values—is what the prover arithmetises. The public values of a shard—the digests of the committed output and of deferred proofs, the program-counter pair and clock on entry and exit, the exit code and shard indices, the address cursors and row counts of the global memory tables, and the septic digest of Section 5.4—are the interface through which consecutive shards and the recursion layers are chained.

3.4 Emulator

Write $\mathsf{Emulate}(P, \mathsf{in}, \mathsf{hint}) = (\mathsf{out}, e)$ for the relation this section defines: the machine of §3 started on the program image P with the two streams available to it, run until it halts, produces the output stream out and the exit status e. It is deterministic: no step reads a clock, a random source or any host state other than the streams. The emulator has two implementations, which produce the record identically: an interpreter and, on Linux x86-64 hosts, a just-in-time compiler that translates MIPS basic blocks to native code with the guest registers pinned to vector-register halves and a flat guest memory whose per-word last-write clock sits next to the word. The two are held bit-exact on every event stream by a differential test in continuous integration, so the compiler is a pure speed-up; §10.2 measures the two rates. When several GPUs prove one execution, a native *producer* in the coordinating process emits, per shard, only a small chunk descriptor (the pre-access values of the memory the shard reads), and the worker that proves the shard re-executes it from that descriptor; the record never crosses a process boundary and the coordinator holds no more than one shard's state (Section 10.2).

4 The core as an intellectual-property block

The sections that follow describe the core from the inside. This one settles what kind of object it is, because that decides what can be measured about it and what the measurements mean.

A silicon core is delivered as a description and characterised on a process. The same description yields a different area, frequency and power on every node, and the description itself is process-independent. The same split organises this paper. The description is a constraint system: the tables, their polynomial identities and the bus interactions between them. The process is a prover that evaluates and commits those tables on a particular accelerator. Every quantity reported here falls on one side or the other, and the two sides behave differently enough that confusing them is the commonest way to misread a result in this area.

Area belongs to the design. The trace area of a unit is its column count times its row count, and both are fixed by the constraint system and the execution rather than by the machine that runs them. It plays the economic role die area plays. Die area determines what a wafer yields and therefore what a part costs; trace area determines how many field elements the prover must commit and open, and therefore what a proof costs. It is not measured in square millimetres and does not convert into them. The consequence is that an arithmetisation change can be stated once, in cells, and is worth the same to every integrator on every accelerator, which is why the per-unit census of Section 10 is given in cells before it is given in seconds. Halving it is this design's counterpart of a shrink.

Throughput belongs to the pair. Proved guest cycles per second is not a property of the core. It is what one accelerator achieves on this core, and a different device, or more of them, moves the figure without altering a single constraint. Three things follow. Scaling has to be reported across device counts rather than as a single number. Figures measured on other projects' hardware are context, not comparison. And the only part of the performance story that transfers between accelerators is the cost model itself, which predicts prover time from area and interaction count and leaves the accelerator to set the constant.

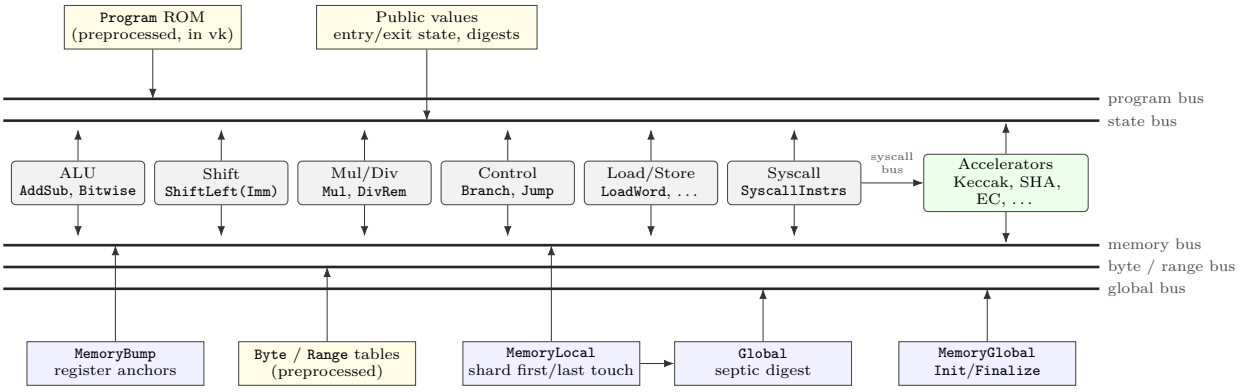


Figure 3: Ziren as a processor. Each executed instruction is one row of one unit; units talk only over buses (program fetch, state chaining, register/memory accesses, byte-table lookups, cross-shard interactions). Preprocessed tables are committed in the verifying key; the public values close the state chain and carry the digests.

Where the analogy stops. There is no process node, no timing closure and no power budget; nothing here has a physical realisation, and the frequency figures describe the prover rather than the part. The analogy also gives something silicon does not. A constraint system is a finite set of polynomial identities over a finite field, so it can be checked mechanically against a specification instead of only simulated against test vectors. Section 9 is that check, and it is the reason this paper reports a verification flow at all rather than a benchmark.

The reference material an integrator needs—the ports, the parameters that may be varied, the ways the core is embedded, and the collateral delivered with it—is collected in Appendix B.

5 Arithmetisation

Figure 3 shows the machine as a block diagram: instruction units connected by buses to a program ROM, a memory subsystem and lookup tables. A unit in the sense of Definition 2.1 has a datapath width, its column count, and an activity, its row count; a *bus* is an interconnect on which units exchange tuples, and the lookup argument of Section 5.8 plays the role of the wiring: it proves that every tuple driven on a bus is consumed exactly once. The area of the design is the sum over units of rows × columns (*trace cells*); Section 10 reports it per unit, as a gate-count report would.

This section is the block's internal design, in the sense of Section 4: the unit widths of Table 3 are its per-unit area figures, the buses are its interconnect, and the decisions taken here—one row per instruction, one shared memory argument, one lookup and one zerocheck—are what fix the area that Section 10 measures. One part of it is configuration rather than design: the accelerator set determines which precompile units a shard can contain, and adding one adds a unit and changes the verifying keys (§B.2).

A shard is proved as a set of units. A unit is a table over $\mathbb{F}$ with a fixed width, a set of polynomial constraints of degree at most three on each row (the constraints are local to a row—the proof system evaluates them at a random point of the multilinear domain, Section 5.9—so relations between consecutive rows are expressed as bus interactions), and a set of interactions on named buses. Every relation *between* units, and every relation between one row of a unit and another, is an interaction. This section describes the unit set, the instruction frame shared by all instruction units, the register and memory argument, the lookup argument and the zerocheck.

5.1 Unit set

Table 3 lists the units of the core machine with their widths. Three families dominate the trace area of an Ethereum block: the memory-instruction units (`LoadWord`, `StoreWord`, `MemoryUnaligned`, `LoadNarrow`, `StoreNarrow`), the ALU units, and the `Global` unit that carries the cross-shard memory argument. The immediate forms of the ALU instructions (`ADDIU`, `ANDI`, `SLTI`, shifts by a constant, ...) have units separate

Table 3: Core-machine units and main-trace widths (columns over $\mathbb{F}$). Preprocessed widths in parentheses. The immediate (`Imm`) variants take the second operand from the instruction word.

Family	Units (width)
Integer ALU	`AddSub` (36), `AddSubImm` (33), `Bitwise` (37), `BitwiseImm` (33), `Mul` (75), `DivRem` (164), `Lt` (50), `LtImm` (47), `CloClz` (93)
Shifts	`ShiftLeft` (62), `ShiftLeftImm` (45), `ShiftRight` (89), `ShiftRightImm` (83)
Control flow	`Branch` (56), `Jump` (57)
Memory instructions	`LoadWord` (49), `LoadNarrow` (60), `StoreWord` (53), `StoreNarrow` (56), `MemoryUnaligned` (60)
Misc. / syscall	`MiscInstrs` (433), `SyscallInstrs` (68), `SyscallCore` (11), `SyscallPrecompile` (11)
Memory argument	`MemoryLocal` (56), `MemoryBump` (13), `MemoryGlobalInit` (152), `MemoryGlobalFinalize` (152), `Global` (58)
Tables	`Program` (1, +14), `Byte` (10, +12), `Range` (1, +2)
Precompiles	SHA-256 extend/compress and controls, `KeccakSponge` (2637) and control, `Poseidon2Permute` (573), Weierstrass add/double/decompress $\times$ 4 curves, Ed25519 add/decompress, $\mathbb{F}_p/\mathbb{F}_{p^2}$ ops, `Uint256MulMod` (441), `U256XU2048Mul` (2773), garbled gate

from the register forms: an immediate row has one register read fewer and a narrower frame, and the split saves roughly a tenth of the width of those rows.

Row counts are rounded up to a multiple of 32 rather than to a power of two; the jagged commitment of Section 6 makes the padding cost proportional to the padding, not to the next power of two. Padding rows have $\mathsf{is\_real} = 0$; every constraint and every interaction multiplicity is gated so that padding rows are vacuous.

5.2 The instruction frame

Earlier designs (SP1 before its hypercube release, and the first generation of Ziren) had a *CPU table* with one row per executed cycle that fetched the instruction, performed the register accesses, chained the program counter, and forwarded a decoded instruction to an opcode unit over a bus; the opcode unit then had a second row per instruction. The current design removes the central table: each instruction unit embeds an *instruction frame* and performs those duties itself, so one executed instruction is exactly one row of one unit. The frame consists of:

- **Program fetch.** The row sends $(\mathsf{pc}, \mathsf{opcode}, \mathsf{op\_a}, \mathsf{op\_b}, \mathsf{op\_c}, \mathsf{imm\_b}, \mathsf{imm\_c})$ on the *program* bus; the preprocessed `Program` table receives it with the multiplicity recorded at trace generation. The instruction word therefore comes from the program image committed in the verifying key, and the unit's opcode selector is checked against it.
- **Register accesses.** Reads of $\mathsf{op\_b}$ and $\mathsf{op\_c}$ (unless immediate) and the read-modify-write of $\mathsf{op\_a}$ at the sub-cycle positions 1..4 of the instruction's tick, through the register variant of the memory argument (Section 5.3). A write to register 0 is committed as zero: the frame carries an $\mathsf{op\_a\_0}$ flag and pins the written word to zero when it is set, so units bind their result through a $(1 - \mathsf{op\_a\_0})$ factor rather than a gate.
- **State bus.** The row receives the tuple $(\mathsf{shard}, \mathsf{clk}, \mathsf{pc}, \mathsf{next\_pc})$ and sends the tuple $(\mathsf{shard}, \mathsf{clk} + \Delta, \mathsf{next\_pc}, \mathsf{next\_next\_pc})$, where the increment Δ is one for an ordinary instruction and a larger bounded value for a syscall, so that the accelerator's memory accesses fit between the calling instruction and its successor (Section 3); the public-values AIR sends the shard's entry state and receives its exit state. Lemma 5.1 is what this buys: the multiset balances only if the rows form a single chain, which is the shard's execution.
- **Clock.** clk is witnessed as a 16-bit and a 9-bit limb, both range checked, so that clock differences in the memory argument can be decomposed cheaply.

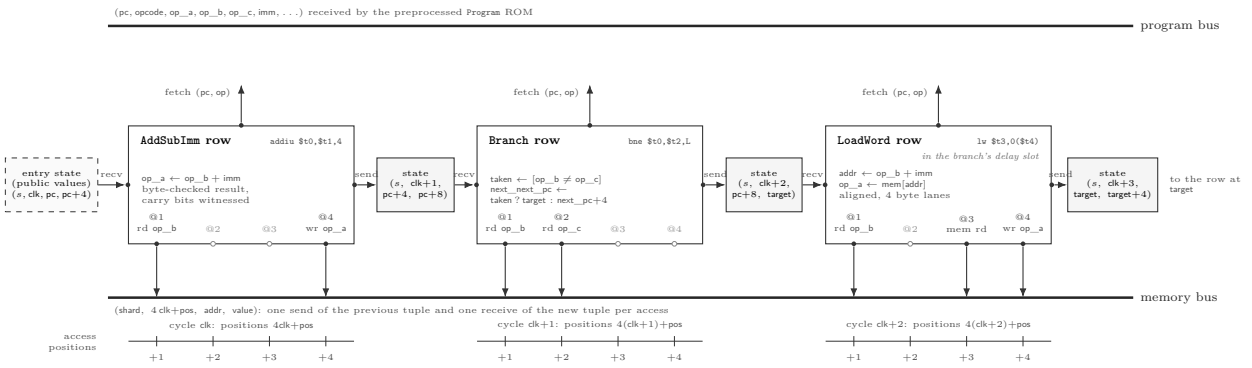


Figure 4: Three consecutive cycles as rows of three units, drawn as a circuit. Each row is a unit with a fetch pin on the program bus, a receive and a send pin on the state bus (the shaded state registers carry (shard, clk, pc, next_pc) from the entry state onward), and four access pins at the sub-cycle positions 4 clk+1..4 on the memory bus; filled pins are the accesses the opcode makes (register reads at 1–2, memory at 3, the register write at 4), hollow pins are unused. The text inside each unit is what its constraints compute. The `LoadWord` row sits in the branch's delay slot: its successor is the branch target, which arrives as data in the state register rather than as pc+4.

Lemma 5.1 (State-bus chain)**.** *In a valid shard (Definition 2.3) with entry state s_0 and exit state s_N, the real instruction rows can be ordered $r_1, \ldots, r_N$ so that r_i receives s_{i-1} and sends s_i, with $\mathsf{clk}(s_i) > \mathsf{clk}(s_{i-1})$; every real instruction row is in the chain, and N is the number of real instruction rows.*

Proof. Each real instruction row receives exactly one state tuple and sends exactly one, with a strictly larger clock, and the public-values table sends s_0 and receives s_N. Balance of the state bus means the multiset of sent tuples equals the multiset of received tuples. Consider the directed graph whose vertices are the distinct state tuples and whose edges are the rows (from received tuple to sent tuple). Every vertex other than s_0 and s_N has in-degree equal to out-degree, s_0 has out-degree one more than in-degree and s_N the reverse, so the edge multiset decomposes into one $s_0 \to s_N$ path and cycles. A cycle is impossible because the clock strictly increases along every edge. Hence the rows form a single path from s_0 to s_N. □

Figure 4 shows three consecutive rows and the buses that bind them. Three frame variants exist: the I-type frame (an immediate as a 4-byte word, used by memory and branch units), the R-type frame (two register reads), and a *shamt* frame for shifts by a constant, whose immediate is a single scalar. The frame byte-checks the word written to op_a; this is the single place where a register value is range checked, and it is what lets memory-instruction rows omit byte checks on the words they move (Section 5.3).

5.3 Registers and memory

Registers and memory share one offline memory-checking argument [BEG+94] in the multiset style used by SP1 [Suc24]: the state of address a is the tuple $(\mathsf{shard}, \mathsf{clk}, a, \mathsf{value})$ of its last access. An access at time (s, c) *sends* the previous tuple (s', c', a, v') and *receives* the new tuple (s, c, a, v) on the *memory* bus, with $v = v'$ for a read. The multiset balances if and only if every access consumes exactly the tuple produced by the previous access to the same address, provided timestamps strictly increase along each address's chain; the AIR of every access enforces $(s', c') < (s, c)$ by witnessing $c - c' - 1$, or $s - s' - 1$ when the shards differ, in range-checked limbs that cover the 27 bits of a $4\,\mathsf{clk} + \mathsf{pos}$ timestamp at a 25-bit clock (a one-bit `compare_clk` selector chooses the branch). Words are four byte columns; the argument carries them as such.

Lemma 5.2 (Memory argument)**.** *In a valid shard, for every address a the accesses to a form a chain ordered by* $(\mathsf{shard}, \mathsf{clk})$ *in which each access consumes exactly the tuple produced by the previous access to a, starting from the tuple supplied by the initial-state row of a and ending at the tuple consumed by its final-state row; in particular every read returns the value of the last write.*

Proof. Each access sends its previous tuple and receives its new tuple on the memory bus, and its constraints enforce that the new timestamp strictly exceeds the previous one. Balance of the memory bus, restricted

to the tuples with address a, gives a graph as in Lemma 5.1 with the initial-state row as source and the final-state row as sink; strictly increasing timestamps exclude cycles, so the accesses form one path, and along the path a read carries its previous value forward unchanged. □

Registers. The `MemoryBump` unit inserts, for every register at its first touch in a shard, a shadow read at $(\mathsf{shard}, 0)$. Since clk restarts at zero in every shard and real accesses sit at positions 1..4 of a tick, the previous access of a register is always in the current shard. The register variant of the access columns therefore drops $\mathsf{prev\_shard}$, the compare selector and one limb, which is re-derived linearly from the others: 10 columns become 7 on every register access of every instruction.

Shard-local memory. Within a shard, the first access to a word must consume a tuple that no access in the shard produced, and the last access produces a tuple that nothing in the shard consumes. The `MemoryLocal` unit records, for every word touched in the shard, its initial and final $(\mathsf{shard}, \mathsf{clk}, \mathsf{value})$; it sends the initial tuple on the memory bus (so the first access can consume it) and receives the final one, and it forwards both as *global* interactions (Section 5.4) so that they are matched across shards. On a `reth` block (the Ethereum execution-client workload of Section 10) a few hundred thousand words are touched per shard, most of them the hot working set touched again in the next shard; the two global rows per touched word are the largest single contribution to trace area, which is why the shard size matters (Section 10).

Whole-execution memory. `MemoryGlobalInit` carries the memory's initial contents (the program image and the hint data written into untouched words) and `MemoryGlobalFinalize` its final contents. Both witness the word as 32 boolean columns, which is what makes every value that enters memory byte-shaped by construction; addresses are 32-bit decomposed and proved strictly increasing along the table through a running comparison against the previous address, with the address cursors published as public values so the recursion can check that consecutive tables tile the address space without overlap. The program image itself is bound by the verifying key: its digest (Section 5.4) is a public parameter, and the init table must consume it.

What is byte-checked. Every value that enters a register is byte-checked by the frame at the write, and every value a precompile writes to memory is byte-checked or bit-decomposed by the precompile unit; memory-instruction rows therefore omit the two additional byte lookups per word that accesses otherwise carry.

5.4 Cross-shard interactions: the global digest

Interactions whose two sides live in different shards—the initial and final tuples of every shard-touched word, the memory init and finalize tables, and syscall pairings—cannot be balanced by a per-shard fraction sum. Ziren carries them in an elliptic-curve digest of the kind SP1 introduced [Suc24], with one deliberate departure. An interaction tuple $\boldsymbol{m} \in \mathbb{F}^7$ together with a one-byte kind is mapped to a point on a curve over the degree-seven extension $\mathbb{F}_{p^7}$ (a *septic* point), sends contribute the point and receives its negation, and the sum of all points over the whole execution must be the identity (offset by a fixed public point so that no running sum is ever the identity). The verifier of a shard proof learns the shard's running sum from its public values, and the verifier of the whole proof adds the shards' sums and the program-image digest from the verifying key.

The departure is in the map. SP1 hashes the message to the curve with a permutation constrained inside the unit, so its points are modelled as random-oracle outputs. Ziren applies no hash: the message *is* the x-coordinate, up to a prover-chosen byte in the top limb, which is what reduces the unit from a few hundred columns to a few dozen. The security argument therefore changes shape, and §5.5 states it in the form the map actually requires.

The `Global` unit has one row per such interaction. It receives the tuple on the global bus, witnesses the curve point $(x, y) \in \mathbb{F}_{p^7}^2$ with $x_6 = 256\, m_6 + \mathsf{offset}$ for a byte offset chosen by the prover so that the cubic has a root (the *lift*), checks $y^2 = x^3 + 3\zeta x - 3$ in $\mathbb{F}_{p^7} = \mathbb{F}[\zeta]/(\zeta^7 + 2\zeta - 8)$, and fixes the sign of y from its top limb: receives have $y_6 \in [1, 63 \cdot 2^{24}]$ and sends $y_6 \in [2^{30} + 1, p - 1]$, two disjoint bands, proved by three byte-table

lookups on the limbs of $y_6 - 1$ (respectively $y_6 - 2^{30} - 1$). The running sum is carried by an *accumulation bus*: row i receives (i, S_i) and sends $(i+1, S_{i+1})$ with $S_{i+1} = S_i + P_i$ enforced by the chord identities of the curve addition, and the public-values AIR closes the chain at the shard's global_cumulative_sum. The unit is 58 columns wide: a 7-limb message, the kind, the point (14), the offset and three range limbs, two running-sum points (28) and the index and flags.

5.5 Security of the digest

Let E be the curve $y^2 = x^3 + 3\zeta x - 3$ over $\mathbb{F}_{p^7}$ and $\mathsf{enc} : \mathbb{F}^7 \times \{0, \ldots, 255\} \to E$ the lift above. Write $\mathcal{M}$ for the set of *reachable* messages: those an interaction of a satisfying trace can carry, given the range facts every sending unit enforces on its tuple. $\mathcal{M}$ is what the security of the digest rests on, and it is much smaller than $\mathbb{F}^7$: each limb of a memory or syscall tuple is a byte, a bounded clock limb or a bounded address.

Lemma 5.3 (injectivity of the lift)**.** *Assume the message limbs satisfy the range facts the sending units enforce, so that* $256\, m_6 + \mathsf{offset} < p$*. Then* enc *is injective on messages: limbs* $x_0, \ldots, x_5$ *are* $m_0, \ldots, m_5$*, and* x_6 *is* m_6 *shifted above a low byte holding the offset, which the bound makes separable; the two sign bands for* y_6 *are disjoint, so* $\boldsymbol{m}$ *and the offset are both recoverable from the point.*

Lemma 5.3 is what makes the digest a faithful encoding of the multiset; it is not by itself what makes the argument sound. If the sends and receives of a claimed execution are the multisets $S \neq R$ and their digests agree, then $\sum_i c_i\, \mathsf{enc}(\boldsymbol{m}_i) = O$ for the non-zero integer vector $c = \mathrm{mult}_S - \mathrm{mult}_R$ over the distinct tuples. Soundness of the cross-shard argument is therefore exactly the following assumption.

Assumption 5.4 (no reachable vanishing combination)**.** For the parameters of a block proof—at most N_{int} interactions in a proved execution, coefficients bounded by the multiplicity bound—it is infeasible to find a non-zero integer vector c supported on at most N_{int} reachable messages with $\sum_i c_i\, \mathsf{enc}(\boldsymbol{m}_i) = O$ in $E(\mathbb{F}_{p^7})$.

Two things should be said plainly about this assumption, because it is the weakest link in the accounting and the one place where our design departs from the construction it descends from. First, SP1 hashes the message to the curve before lifting it, which lets the points be modelled as random-oracle outputs; the corresponding assumption is then a clean discrete-logarithm statement about points no adversary can steer. We removed the hash, so our points are chosen by the prover, and Assumption 5.4 must be read over a structured set rather than a random one. The protection is the *sparsity* of $\mathcal{M}$: a prover who could realise an arbitrary point could pick a generator G and scalars k_i summing to zero, read the x-coordinates off as messages, and forge at will. That attack is blocked exactly to the extent that every sending unit bounds every limb it contributes, and collecting those bounds into one statement is an obligation of the arithmetisation rather than of the protocol.

Second, sparsity is not the end of the analysis. The relevant attack on a subset-sum in a group is the generalised birthday method, whose cost falls as the adversary is allowed more lists; the bound must therefore be taken against the number of interactions a prover can actually place in a trace, not against the group order alone. The generic-group cost of a single discrete logarithm in the large prime-order subgroup of $E(\mathbb{F}_{p^7})$ is the square root of that subgroup's order, and $|E(\mathbb{F}_{p^7})| \approx p^7 \approx 2^{217}$; we do not state a bit figure here because the honest figure is the one that comes out of the k-list analysis at our interaction count, and that analysis is not yet done. This is a computational assumption, not the information-theoretic bound the rest of the accounting enjoys, and Section 7.3 lists it as a separate component.

Exceptional cases of the chord addition (doubling and the identity) cannot occur on honest traces because the running sum starts at a fixed public offset point and the fixed offset point, written D, is chosen with $\pm D$ outside the image of enc, and a dishonest trace that hits one fails the addition identities.

5.6 Trace uniqueness

Theorem 5.5 (From unit determinism to trace uniqueness)**.** *Fix a program image, public values and a valid shard. If every unit of the shard is deterministic in the sense of Definition 2.4, then the rows reachable from the entry state along the state and memory buses are determined by the program image and the public values: any two valid shards with the same program image and public values agree on those rows up to permutation.*

Proof. By Lemma 5.1 the instruction rows of either shard form one chain from the entry state; write $r_1, \ldots, r_N$ for the first shard's chain. We show by induction on i that r_i has the same inputs and outputs in both shards. The inputs of r_i are its received state tuple s_{i-1}, its fetched instruction, and the previous values of the addresses it accesses. s_0 is public and s_{i-1} is the output of r_{i-1} by induction. The fetched instruction is a function of $\mathsf{pc}(s_{i-1})$ because the program bus is received only by the preprocessed program table, which is committed. The previous value of an address is, by Lemma 5.2, the value written by the last earlier access to that address, or the initial value from the memory-initialisation table, which is bound to the program image and the public address cursors; earlier accesses belong to rows r_j, $j < i$, whose outputs agree by induction. So the inputs of r_i agree, and determinism of its unit gives that its outputs agree. Rows of non-instruction units (the memory-initialisation and finalisation tables, the shadow reads, the digest rows, the tables) are determined in the same way from the bus tuples they receive, which are outputs of instruction rows or public. Rows that participate in no bus reachable from the entry state are not covered by this argument; the arithmetisation admits none, because every real row carries a frame or a memory access, but that is a property of the unit set rather than of the buses, and we do not prove it here. □

The theorem is stated, not machine-checked; the unit-level determinism statements are its machine-checked premises (Section 9.2), and the two bus lemmas are the standard arguments of the multiset construction.

5.7 Preprocessed tables

The `Byte` table has 2^{16} rows indexed by a byte pair and one multiplicity column per operation (bitwise operations, shifts, comparisons and 8/16-bit range checks); every carry, borrow, comparison and range check in the machine is an interaction with it. The `Range` table serves the clock limbs, and the `Program` table holds the decoded program image. Preprocessed tables are committed at setup as part of the verifying key and opened at the same points as the main trace.

5.8 The shard-level lookup argument

All interactions of a shard—on the program, state, memory, byte, range, syscall, global and accumulation buses—are proved by *one* LogUp-GKR instance. After the trace is committed, the verifier samples $\alpha, \beta \in \mathbb{F}_{p^4}$; every interaction $(\mathsf{bus}, \boldsymbol{f}, m)$ of every unit contributes the fraction $m/(\alpha - \mathsf{bus} - \sum_{j \geq 1} \beta^j f_j)$, and the prover shows that the sum over all sends equals the sum over all receives. The fractions form the leaves of a GKR circuit of $\log_2$(leaves) layers, each layer adding pairs of fractions; layer transitions are proved by sumcheck from the root down, and the final layer's claims are evaluation claims on the columns that enter the fractions—multilinear evaluations of unit columns at a random point, which are handed to the polynomial commitment. On a reth block a shard has of the order of 10^8 leaves, and Section 10 shows that this circuit is the single largest consumer of GPU time; the number of leaves is the number of (row, interaction) pairs, so interaction count per row is a cost in its own right, distinct from column count.

Soundness of the fraction identity is $\deg / |\mathbb{F}_{p^4}|$ per sumcheck round plus the collision probability of the fingerprint, which grows with the number of leaves; Section 7.3 lists it as one component of the accounting. No grinding is applied to the lookup challenges in the current schedule.

5.9 The shard-level zerocheck

The polynomial constraints of all units are proved by *one* zerocheck. The verifier samples λ for a random linear combination across units and μ across the constraints of a unit; for each unit the polynomial $C_c(\boldsymbol{X}) = \sum_k \mu^k \, c_{c,k}(\mathrm{row}(\boldsymbol{X}))$ must vanish on the unit's real rows. The prover proves $\sum_{\boldsymbol{b}} \mathsf{eq}(\boldsymbol{b}, \boldsymbol{r}) \sum_c \lambda^c C_c(\boldsymbol{b}) = 0$ for a random $\boldsymbol{r}$ by sumcheck over the tallest unit's variables, with shorter units handled by an analytic "virtual $\geq$" factor rather than padding, and the final evaluation claims on the units' columns are again handed to the commitment. The constraint degree is three, which fixes the degree of the round polynomials at four; the constraint-evaluation kernel on the GPU is compiled per unit from the same constraint description that the verifier uses, except for the widest accelerator units, which are interpreted (Table 5), and a mismatch between the two is caught by host verification (Section 10).

The zerocheck's claims are seeded from the LogUp-GKR openings so that the two protocols share one opening of each column: after the two sumchecks, what remains is a set of (unit, column, point, value) claims,

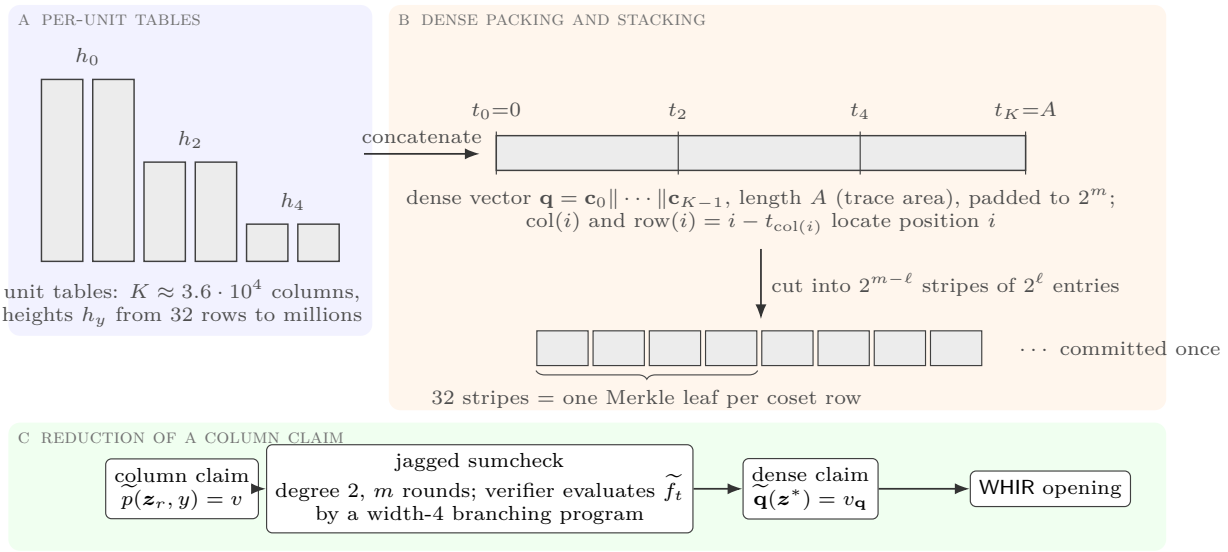


Figure 5: From unit tables of different heights to one committed polynomial: columns are concatenated into the dense vector, which is cut into stripes of the stacking height and committed once; a column claim is turned into a claim on the dense vector by the jagged sumcheck.

of the order of $3.6 \cdot 10^4$ per shard, and turning those into a single claim on a single committed polynomial is the job of the commitment layer.

6 The polynomial commitment: stacking and the jagged reduction

After the lookup argument and the zerocheck, the verifier holds evaluation claims on individual unit columns. A shard of the `reth` workload has about $3.6 \cdot 10^4$ columns whose heights range from 32 rows to millions; committing each column, or each unit, separately would cost the verifier a Merkle root and a query phase per commitment, and the recursion circuit would depend on which units a shard contains. Ziren commits the whole shard as *one* multilinear polynomial and reduces the column claims to a claim on it. The reduction has two steps, drawn in Figure 5: a *stacking* that turns the concatenation of all columns into a matrix of fixed height, and the *jagged* sumcheck of [HJR+25] that maps a claim about a column's own coordinates to a claim about its position in the concatenation.

The property this buys is modularity in the sense of Section 4: because the verifier of the reduction depends only on the logarithm of the trace area, a unit can be widened, narrowed, added or removed without changing the shape of the program that verifies the shard. The block's internals can be revised, as they were four times in Section 10.4, without redesigning the layer above them.

6.1 Dense packing and stacking

Let the columns of a shard be $\mathbf{c}_0, \ldots, \mathbf{c}_{K-1}$ with heights $h_0, \ldots, h_{K-1}$ (unit order, then column order within a unit), and let $t_k = \sum_{j<k} h_j$ be the prefix sums. The *dense vector* is $\mathbf{q} = \mathbf{c}_0 \| \mathbf{c}_1 \| \ldots \| \mathbf{c}_{K-1}$, of length $A = t_K$, the *trace area*. The prover pads $\mathbf{q}$ with zeros to length 2^m, $m = \lceil \log_2 A \rceil$, and views it as the multilinear $\widetilde{\mathbf{q}}$ on m variables.

The commitment layer does not see $\widetilde{\mathbf{q}}$ as one vector of 2^m entries but as a matrix: a *stacking height* 2^ℓ is fixed by the configuration, the vector is cut into $2^{m-\ell}$ stripes of 2^ℓ consecutive entries, and the stripes are grouped into the leaves of the Merkle commitment, 32 to a leaf in the deployed configuration (Section 7.2). A point $\boldsymbol{z} \in \mathbb{F}_{p^4}^m$ splits as $(\boldsymbol{z}_{\mathrm{stripe}}, \boldsymbol{z}_{\mathrm{row}})$ with $|\boldsymbol{z}_{\mathrm{row}}| = \ell$; the evaluation $\widetilde{\mathbf{q}}(\boldsymbol{z})$ equals the interpolation, by the Lagrange weights of $\boldsymbol{z}_{\mathrm{stripe}}$, of the per-stripe evaluations at $\boldsymbol{z}_{\mathrm{row}}$, which is how the WHIR opening of the stripes is bound to the claim on $\widetilde{\mathbf{q}}$. Before anything is committed, the per-unit dimensions (name, row count,

column count, prefix sum) are absorbed into the transcript, so a prover cannot re-interpret the packing after seeing challenges.

6.2 The jagged reduction

Following [HJR+25], view the shard's trace as a *jagged function* $p : \{0,1\}^n \times \{0,1\}^k \to \mathbb{F}$ on rows x and columns y, zero for $x \geq h_y$, with $n = \max_y \lceil \log_2 h_y \rceil$ and $2^k \geq K$. Its dense representation is exactly $\mathbf{q}$: writing $\mathrm{col}(i)$ for the column containing position i and $\mathrm{row}(i) = i - t_{\mathrm{col}(i)}$,

$$\widetilde{p}(\boldsymbol{z}_r, \boldsymbol{z}_c) \;=\; \sum_{i \in \{0,1\}^m} \mathbf{q}(i)\, \mathsf{eq}(\mathrm{row}(i), \boldsymbol{z}_r)\, \mathsf{eq}(\mathrm{col}(i), \boldsymbol{z}_c) \;=\; \sum_i \widetilde{\mathbf{q}}(i)\, \widetilde{f}_t(\boldsymbol{z}_r, \boldsymbol{z}_c, i),$$

where $f_t(\boldsymbol{z}_r, \boldsymbol{z}_c, i) = 1$ iff $\mathrm{row}(i) = \boldsymbol{z}_r$ and $\mathrm{col}(i) = \boldsymbol{z}_c$, and $\widetilde{f}_t$ is its multilinear extension. A column claim "column y evaluated at $\boldsymbol{z}_r$ equals v" is the claim $\widetilde{p}(\boldsymbol{z}_r, y) = v$, so all column claims of the shard are claims on $\widetilde{p}$; the verifier batches them with a random γ into one claim $\sum_c \gamma^c \widetilde{p}(\boldsymbol{z}_r^{(c)}, y_c) = \sum_c \gamma^c v_c$ (the batching of Section 5 of [HJR+25]), and the prover and verifier run the sumcheck of the product $\widetilde{\mathbf{q}}(i) \cdot \sum_c \gamma^c \widetilde{f}_t(\boldsymbol{z}_r^{(c)}, y_c, i)$ over $i \in \{0,1\}^m$. This is a degree-two sumcheck of m rounds; at its end the verifier holds one claim $\widetilde{\mathbf{q}}(\boldsymbol{z}^*) = v_{\mathbf{q}}$, which goes to WHIR, and one claim on $\widetilde{f}_t$ at $(\boldsymbol{z}_r, \boldsymbol{z}_c, \boldsymbol{z}^*)$, which it must evaluate itself. The prover's work is a small constant number of field multiplications per entry of $\mathbf{q}$: the paper's bound is $5 \cdot 2^m + 2^n + 2^k$ multiplications for the reduction, on top of the commitment.

Evaluating $\widetilde{f}_t$. The verifier must evaluate $\widetilde{f}_t$ itself. Proposition 3.2 of [HJR+25] writes it as a sum over columns of a function computed by a width-four read-once branching program [Bar89], whose multilinear extension is evaluable in $O(m)$ field operations by the dynamic program of Holmgren–Rothblum [HR18]; the sum over the $K \approx 3.6 \cdot 10^4$ columns is delegated to the prover by the "jagged assist" sumcheck of the same paper. Appendix A.2 gives the formulas as implemented. The verifier's cost is $O(m)$ field operations plus the sumcheck rounds, and it is an arithmetic circuit that depends only on m: the recursion program for a shard is a function of the shard's log-area class, not of its unit set.

Soundness. The reduction adds $m/|\mathbb{F}_{p^4}|$ for the product sumcheck and $2(m+1)/|\mathbb{F}_{p^4}|$ for the assist, both negligible at $|\mathbb{F}_{p^4}| \approx 2^{124}$; it appears as the "reduce to dense PCS" component of the accounting in Section 7.3.

6.3 Instantiation on the inner and outer rings

The same jagged layer is instantiated twice: on the inner ring, where core and compress shards are committed with WHIR over KoalaBear (Section 7), and on the outer *wrap* ring, where the final recursion shard is committed with the FRI-based BaseFold-style instantiation Section 8 describes. The stacking, the dense packing and the jagged sumcheck are shared; only the opening of $\widetilde{\mathbf{q}}(\boldsymbol{z}^*)$ differs.

7 The proximity test: WHIR over KoalaBear

The claim $\widetilde{\mathbf{q}}(\boldsymbol{z}^*) = v$ on the stacked polynomial is proved with WHIR [ACFY24b], an interactive oracle proof of proximity (IOPP) for *constrained* Reed–Solomon codes, that is, for codewords whose multilinear also satisfies a stated weighted sum. We recall the protocol in the form it is implemented, then give the commitment layout and the soundness accounting. This layer holds the block's principal configuration knob (§B.2): at a fixed soundness regime the schedule trades proof size against prover time, and an integrator who wants a smaller proof pays for it here. The accounting below, and the report it generates for a given configuration, are part of the collateral of §B.4.

7.1 The protocol

WHIR tests membership of a function $f : L \to \mathbb{F}$ in $\mathsf{CRS}[\mathbb{F}, L, m, \widehat{w}, \sigma]$ (Section 2). One iteration with folding factor k runs k sumcheck rounds, sends a folded oracle re-encoded at the round's rate, answers out-of-domain

samples, answers t shift queries by opening 2^k symbols each, and recurses on a new constrained code with k fewer variables; after the last iteration the final polynomial is sent in the clear. Appendix A.1 recalls the iteration as implemented. The out-of-domain samples are what distinguish WHIR from BaseFold: they pin the folded oracle to a unique nearby codeword before the shift queries are drawn, so the per-query soundness contribution is governed by the proximity parameter of the regime, which at a given rate is what the out-of-domain step pins down. The first prover of this generation used BaseFold at a fixed rate with a full query set in every round; WHIR at the same provable level uses fewer queries in its later rounds and a handful of committed rounds rather than one per variable, and shares the prover-side skeleton (encode, Merkle-commit, fold), so the switch required no new kernels beyond the out-of-domain evaluation and the monomial-basis weights.

7.2 Commitment layout

Ziren uses the *interleaved* commitment: a polynomial in v variables is reshaped into a $2^{v-k_0} \times 2^{k_0}$ matrix whose column c is the stride-2^{k_0} slice $f[h \cdot 2^{k_0} + c]$, each column is Reed–Solomon encoded independently (coefficients zero-padded, one DFT per column), and, for a single stacked stripe, Merkle leaf j holds the 2^{k_0} column values at domain point x_j. Folding a leaf by the round's challenge $\boldsymbol{r}$ gives the monomial evaluation of the folded polynomial at x_j, so a query answer enters the next sumcheck as an ordinary constraint whose weight folds in closed form. The stacked stripes of Section 6.1 are committed with this layout: the stripes are grouped 32 to a leaf, so a leaf of the first committed round authenticates one coset row from each of them, 32×2^{k_0} field elements. The folding factor k_0 of the first committed round is therefore chosen small, because the leaf width multiplies every query of that round and re-hashing those leaves is the recursion verifier's dominant cost; later rounds query a single folded polynomial and fold more variables at a time.

Each committed round re-encodes the folded polynomial with one DFT per column at the new rate, and the opening of a shard of the `reth` workload costs about 39 ms of host-visible time per shard on the GPU with device-resident rounds; the round-0 re-encoding of the stripes is the largest single term. No proof-of-work is spent on the folding challenges; all grinding sits on the query phases, where a bit of work buys a bit of security against the query-sampling adversary.

7.3 Soundness accounting

Round-by-round soundness of the compiled argument is bounded by the *sum* of the errors of the transcript's rounds. Each term is expressed as bits $= -\log_2$ of its error, computed with the `soundcalc` tool [Eth26a] on the configuration file kept with the source, and then the union bound over the terms; the level of a proof is the latter, not the smallest term.

Regimes. In the *unique-decoding* regime (UDR) the proximity parameter is $\delta = (1-\rho)/2$ and a shift query contributes $-\log_2(1-\delta) = -\log_2 \frac{1+\rho}{2}$ bits: at a rate of $1/4$ this is 0.678 bits and it approaches one bit as the rate falls; no conjecture is needed. In the *Johnson* regime $\delta = 1 - \sqrt{\rho} - \eta$ and a query is worth up to $\log_2(1/\sqrt{\rho})$ bits, which the correlated-agreement results of [BSCI+20, ACFY24b] give up to the Johnson bound without conjecture; only the further reach towards capacity, $\delta \to 1-\rho$, is conjectural, but folding terms $\approx (2^m 2^k/\rho)/|\mathbb{F}_{p^4}|$ that do not improve with more queries cap what that regime can add over a degree-four extension of a 31-bit field. Ziren therefore sets its query counts in the unique-decoding regime so that every committed round clears the same per-round target; with t queries and b bits of query grinding a round is worth $t \cdot (-\log_2 \frac{1+\rho}{2}) + b$ bits.

Components and their composition. The transcript's components are: the out-of-domain samples and the shift queries of each round, the batching of the column claims, the final polynomial, each folding step, the reduction to the dense polynomial (Section 6), the zerocheck, the LogUp-GKR argument, and the hash. `soundcalc` evaluates each as a function of the trace length, the column count, the schedule and the field; the shard proof's error is the *sum* of the components' errors, so its level in bits is $-\log_2$ of that sum, which is a few bits below the smallest component whenever several components sit near it. One further term sits outside `soundcalc`'s model and must be composed by the reader: the cross-shard digest of Section 5.4 (Assumption 5.4, a computational assumption over a prover-chosen point set, and the one we regard as the

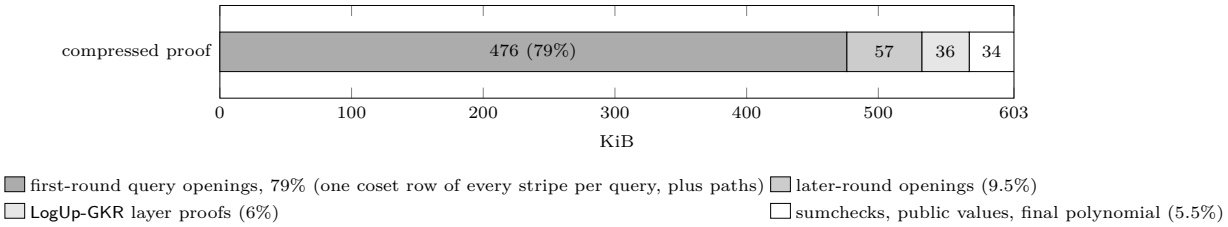


Figure 6: Anatomy of a measured compressed proof (617 618 bytes). Round-0 query openings dominate because each opens one coset row of every stripe.

weakest link). Modelling the Poseidon2 transcript as a random oracle is not a further term but the setting in which the compiled statement holds at all. The configuration file and the generated report for the deployed schedule are part of the artifact; we do not restate the resulting figures here.

Theorem 7.1 (Composition)**.** *Work in the random-oracle model, and let an adversary make at most Q queries to the oracle. Let every proof in the tree—each shard proof, and each leaf and compose proof of the recursion machine—be the Fiat–Shamir compilation [BSCS16] of an interactive protocol that is round-by-round knowledge sound with error $\varepsilon_i = \sum_c \varepsilon_{i,c}$ over the components of its own transcript. Let a block proof be a tree in which N such proofs are each verified once by the node above them, with the root verified by the verifier and every node's verifying key bound by the allowlist of Section 8. Assume further that the extractor of each node succeeds on the transcript its parent's extractor produces, and not only on transcripts drawn from an honest interaction. Then the block proof has knowledge error at most*

$$Q \sum_{i=1}^{N} \varepsilon_i \;+\; \varepsilon_{\mathrm{digest}},$$

where $\varepsilon_{\mathrm{digest}}$ is the error of Assumption 5.4.

Proof sketch. Compiling a round-by-round knowledge sound protocol with Fiat–Shamir costs a factor in the number of oracle queries: an adversary may rewind and retry, and the standard analysis charges Q attempts against the per-round error [BSCS16, CY24]. That accounts for the factor Q and for the fact that the random-oracle model is where the statement lives rather than an additive term in it. Within the tree, the extractor of the compiled argument runs the extractor of each verified proof in turn: the extractor for a compose node yields the transcripts of its children, and so on down to the shard proofs, whose extractor yields the witness rows. The allowlist hypothesis is what makes each step an extraction from the intended verifier; without it a node could verify a program other than the compose or leaf program. The hypothesis about parent-produced transcripts is the step this sketch does not discharge, and it is not free: a child's extractor is invoked on a transcript that the parent's extractor constructed, which is not the distribution against which its guarantee is stated. Given it, the failure events combine by the union bound. □

For the block of Section 10, N is its core shards plus the leaf and compose nodes above them, of the order of two hundred proofs. We state the bound because it is what the union bound gives; published levels for other systems are per proof and do not compose over the tree.

Proof size. The shard proofs of a block total about 112 MB before recursion (§10.4); what is published is the compressed proof at the root of the tree. A measured compressed proof of an Ethereum block is 617 618 bytes (603 KiB), of which 79% is the openings of the first committed round (its queries, each authenticating one coset row of every stripe, plus their Merkle paths), 9.5% the later rounds and 6% the LogUp-GKR layer proofs; the size is, to first order, $t_0 \cdot (\text{stripes}) \cdot 2^{k_0}$, which is why moving from an earlier, more conservative schedule to the deployed one shrank the proof by 38%, from the megabyte of the first row of Table 1 to 603 KiB. Figure 6 draws the anatomy.

8 Recursion and the outer ring

This section describes the block's outward interface: what leaves it is the root of the tree built here, the compressed proof of Table 8, and what binds that proof to the advertised machine is the verifying-key allowlist of the last paragraph. Shard proofs are aggregated by a recursion machine: a small register VM over the same field whose programs are compiled from a description of the verifier, executed to a trace, and proved by the same unit/LogUp-GKR/zerocheck/Jagged/WHIR pipeline. Its units are the arithmetic of $\mathbb{F}$ and $\mathbb{F}_{p^4}$, memory, the Poseidon2 permutation, a select gate and a batched folding gate for the FRI-style query checks.

Leaves. A *leaf* program verifies one core shard proof in four phases mirroring the host verifier: the transcript prologue (public values, commitment, unit dimensions), the LogUp-GKR layer replay, the zerocheck replay including the constraint evaluation of every unit present, and the jagged opening with its branching-program closing identity followed by the WHIR verifier. Because the jagged verifier depends only on the shard's log-area and the constraint evaluation only on the unit set, the number of distinct leaf programs is small; on the single-GPU deployment consecutive shards of the same class reuse a device-resident proving key. Adjacent shards' leaves are *fused* into one node where the shard shapes allow, and leaves are proved on the same GPU concurrently with core shards, so that the GPU's time is shared between the two stages rather than serialised.

Compose tree. Leaf proofs are merged by *compose* programs of arity up to four that verify their children, check that the children's public values chain (exit state of shard i equals entry state of shard $i+1$, the memory-address cursors tile, the septic digests accumulate), and emit the merged public values. The tree merges contiguous shard *ranges* as soon as they are complete rather than level by level, which shortens the tail after the last core shard from four serial composes to two; that tail is one of the two components of the gap to linear scaling in §10.5. Arity is capped at four by the recursion units' height budget.

Shrink and wrap. The root proof is *shrunk* to a fixed shape and then *wrapped* on the outer ring: a jagged proof whose dense polynomial is committed with a FRI-style BaseFold instantiation over the same field but with a 254-bit hash, folding one variable per round under query grinding down to a small final polynomial. Its accounting (same tool, same regime; the LogUp-GKR term is the binding one) and an expected 545 KiB proof size are in the artifact's report. The wrap proof is what a Groth16 verifier on BN254, generated with `gnark` [Con24], consumes when an on-chain proof is required; for the block-proving service, and throughout this paper, the compressed inner-ring proof itself (Section 10) is the published artefact.

Verifying-key binding. Every recursion program has a verifying key. The statement of Section 1.1 is only about the advertised machine if the verifier can tell that the leaf it is looking at verified a *core* proof of that machine and that every compose node ran the compose program: the compose program therefore checks that its children's keys belong to a Merkle allowlist, and the root of that allowlist is checked against the one compiled into the verifier. Which verifier matters: inside the tree the root is a witness the prover supplies, so the in-circuit checks establish only that every child's key lies in a tree with *that* root. The root is therefore carried out of the tree as a public value of the compressed proof and, for the wrapped proof, as a public input of the wrap circuit, where the on-chain verifier supplies the published root rather than accepting one from its caller. It was not always so, and the gap is recorded in Table 9. The recursion machine is outside the extraction of Section 9, so Theorem 7.1 takes this binding as a hypothesis. It is part of the protocol: a proof produced by a prover configured to skip the in-circuit check has a different compose key and is rejected (Section 9 reports finding this configuration in the block-proving service). The allowlist is regenerated whenever the arithmetisation changes; it is enumerated for the recursion programs and collected from executions for the core programs, so it currently covers the block guest rather than an arbitrary program.

9 Design verification

The soundness argument of Sections 5–7 establishes that an accepting proof implies the existence of a trace satisfying every constraint. Two further questions decide whether that trace is an execution of the advertised

machine, and they are the questions asked of a processor before tape-out: does the emulator implement the ISA (functional correctness), and does the constraint system of each unit admit exactly one behaviour for each input (the circuit is deterministic, so a satisfying trace can only be the execution)? We treat both with the tools of hardware verification: a specification-derived conformance suite against an independent reference, an executable formal model, and per-unit uniqueness theorems. What this section produces is the verification collateral of §B.4: the suites and their reference results, the formal model and its two checks, the extracted theorems with the tactic that discharges them and the list of those still open, and the defects register of Table 9. An integrator reads it to decide what has been established once, for every integration, and what has not.

9.1 ISA conformance

Vectors from the specification. The MIPS32r2 architecture manual's encoding tables were transcribed into a machine-readable table of 77 instructions (all integer, shift, branch, jump, load/store, bit-manipulation and conditional-move forms the guest supports). For each instruction a generator emits ten programs with pseudo-random operands, boundary values and the delay-slot placements the instruction admits, assembled with `llvm-mc`; the reference is the Unicorn/QEMU MIPS32r2 emulator [QV15], which shares no code with Ziren's own, and each vector compares the general registers, `HI`/`LO` and the touched memory after execution. All 770 vectors agree, on both the interpreter and the JIT.

Shared instruction suites. The generated vectors cover the encoding space broadly but shallowly. A second suite covers the opposite way: fifteen hand-written suites, 144 cases in all, for the instructions whose MIPS32r2 semantics have the most corners—signed and unsigned division, the multiply-accumulate pairs that read and write `HI`/`LO`, the bit-field and byte-order instructions `SEB`, `SEH`, `WSBH`, `INS` and `EXT`, the rotates, and the count-leading-one and count-leading-zero instructions. Their case data lives in one crate that both the emulator tests and the prover tests consume, so every case is executed and proved against the same expected results. The generated vectors are shared the same way, one file read by both harnesses; between them, a regression that appeared in the emulator but not the prover, or the reverse, would show up as a disagreement rather than as two tests drifting apart.

The Cannon suite. The third suite is independent of us. Optimism's Cannon ships a suite of hand-written MIPS test programs [Opt23], written by the fault-proof project whose ISA semantics a hybrid deployment must match (§B.3); they exercise the machine against another team's reading of the architecture manual rather than against our own generator or our own regressions. A harness byte-swaps them to little-endian and adapts the halt convention; the 49 programs applicable to a little-endian guest all pass.

From vectors to proofs. 224 of the generated programs and 48 of the 49 Cannon programs were also proved end to end and verified. The single exception halts with a non-zero exit code, which the emulator accepts and the arithmetisation does not admit as a valid execution to prove. On everything else the arithmetisation accepts every behaviour the reference exhibits; the next subsection is the converse direction.

Executable formal model. A Lean 4 model of the ISA was written independently of Ziren's emulator: a decoder transcribed from the encoding tables, an interpreter with the delay-slot program-counter pair and little-endian byte memory, and an ALU. Two checks connect it to the implementation, both discharged by `native_decide`: the decoder agrees with the emulator's decoder on all 2 275 distinct instruction words appearing in the vectors, and the model reproduces the reference emulator's results on all 770 vectors. Ziren's two emulator implementations are additionally run twice on every vector with a digest of the emitted event records, which agree between runs and between the interpreter and the JIT; this is evidence of reproducibility rather than a proof of determinism.

9.2 Determinism of the units

What is proved. For a unit U with constraint system C_U over a row w of columns, and with *inputs* $I_U(w)$ (the values the row receives on its buses: the fetched instruction, the register and memory values read, the

incoming state) and *outputs* $O_U(w)$ (the values it sends: the register and memory values written, the outgoing state), determinism is the statement

$$\forall w, w'.\ C_U(w) \wedge C_U(w') \wedge I_U(w) = I_U(w') \ \Rightarrow\ O_U(w) = O_U(w').$$

The statement is Definition 2.4, with byte-table lookups lowered to range constraints or to calls of an abstract deterministic table.

Theorem 5.5 is what makes a satisfying trace unique given the determinism of every unit. The theorems below are its premises, machine-checked for the units the automation closes and stated with an explicit gap for the rest.

Extraction. The theorems are not written by hand. The same Rust constraint description that the prover evaluates and the verifier checks is evaluated once with symbolic columns, the resulting polynomial constraints, range facts and bus interactions are lowered to a first-order statement, and a generator emits one Lean 4 file per unit: a record type for the row, the constraints as a conjunction, the input and output projections with their provenance, and the theorem above. Selector-specialised units (one theorem per opcode selector) and an *is_real* $= 1$ specialisation keep each statement about real rows of one opcode. The extraction covers all 64 units of the core machine: 106 determinism theorems.

Proof automation. A theorem is closed by a tactic that lifts the statement from $\mathbb{F}$ to the integers: every column is replaced by an integer with the bound its range fact gives (a byte, a 16-bit limb, a bit, or $[0, p)$ when unconstrained), every field equation becomes an exact integer equation once its slack is shown to vanish under those bounds, and the remaining goal is linear arithmetic over bounded integers, decided by Lean 4's `omega`. Between the lifting and the decision the tactic runs the propagation loop of Picus inside the proof assistant: it proves corresponding columns of w and w' equal one pair at a time, substitutes, resolves selector bits to constants, and case-splits only on selector bits already known to agree. The automation runs with a fixed wall-clock budget per theorem. At the time of writing 17 of the 106 theorems are closed (Figure 7, by unit family). 22 were attempted and remain open: integer units whose constraints multiply by 2^{24} or 2^{32} and are only meaningful modulo p (multiply, the conditional moves, the memory-argument units), which the integer lifting does not model, or whose selector case structure exceeds the split budget. 41 are not attempted by the generator: 24 are accelerator units above the size threshold (the same theorem at 1 000–5 600 conjuncts), and 17 are units whose constraints contain comparison case-splits from the byte-table summaries, which the current tactic does not handle. 26 had not finished their attempt when the numbers were taken. The syscall unit, which was under-determined as first extracted (next paragraph), closes after the bus change; the count above is therefore for the constraint system with that change applied. Two units separate it from the system measured in Section 10: that one, and the hash unit, whose round schedule was corrected afterwards (Table 9). All theorem statements compile; open ones carry an explicit `sorry` and are listed in the artifact.

9.3 Findings

The flow surfaced nine defects, and how they were found is more informative than what they were. Three came from the determinism theorems, and all three are under-constraint: a value that the constraints permit to take more than one form. Three came from end-to-end proving, and all three are integration mistakes: two components that each behave correctly and disagree about a convention. The last three came from reading this implementation line by line against the RISC-V system it descends from, and they share a property the other two techniques cannot reach. Each is an *omission*—a check that is absent rather than wrong.

That distinction is worth stating plainly, because it bounds what the first two techniques can establish. A determinism theorem asks whether the constraints present admit two behaviours; it is silent about a constraint that was never written, since its absence makes no theorem false. Testing is silent for the same reason: an omitted check breaks nothing that an honest execution does, so no test fails. The completeness flag left unpinned at the terminal stages is the sharpest example. Every honest proof sets it, so every test passes and every extracted theorem holds; only a reader asking what enforces it finds that nothing does. Differential reading against an independent implementation of the same protocol is the cheapest way we know to find that class, and it is the reason we report it as a third technique rather than as incidental review.

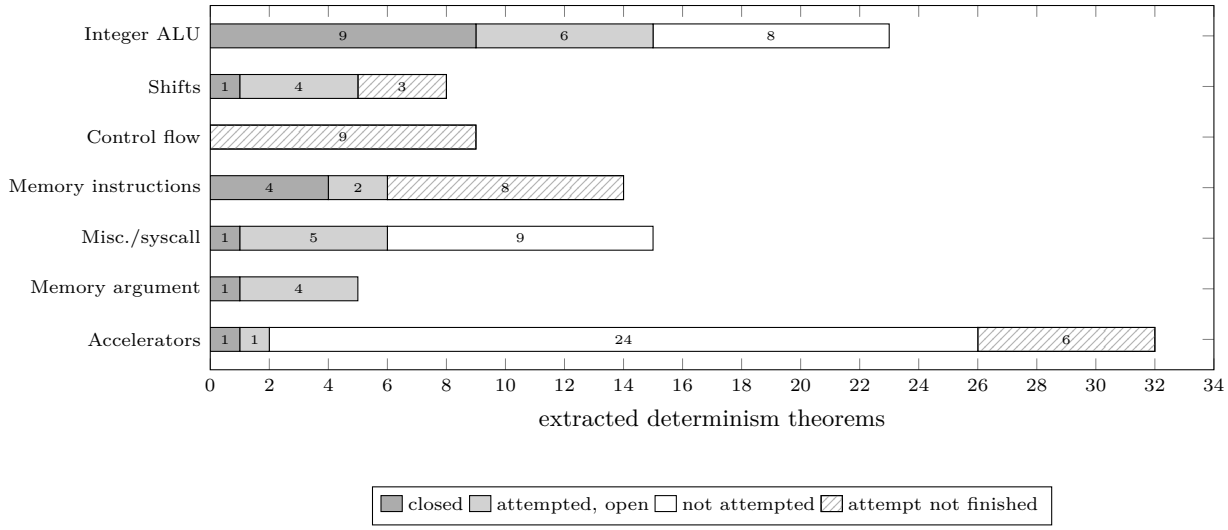


Figure 7: Status of the 106 extracted determinism theorems (one per unit and opcode selector), by unit family. "Not attempted" theorems are skipped by the generator: accelerator units above the size threshold and units with comparison case-splits. "Attempt not finished" theorems were still under the automation's budget when the numbers were taken.

Appendix C tabulates all nine. One is described in detail here, because it is the one the extracted theorems found unaided and it shows what such a theorem does and does not say.

Syscall argument half-words. The syscall unit receives a reduced 32-bit argument $a \in \mathbb{F}$ from the instruction that raised the syscall and witnesses it as two 16-bit half-words $a_{\mathrm{lo}}, a_{\mathrm{hi}}$ constrained by $a_{\mathrm{lo}} + 2^{16} a_{\mathrm{hi}} = a$ in $\mathbb{F}$ and two 16-bit range checks. Because $2^{32} > p$, this does not determine the half-words: for every $a \in \mathbb{F}$ both the decomposition of a and that of $a + p$ satisfy the constraints, since $2p < 2^{32}$. The extracted theorem for the unit is false and the automation reports exactly these four outputs as open. Whether the ambiguity is exploitable depends on the consumer: an accelerator that re-reduces the half-words modulo p sees the same field element either way, while one that uses them as an exact 32-bit pointer does not. The finding is a specification gap between the two sides of the syscall bus. The remedy, applied after the finding, is to carry the half-words rather than the reduced sum on the bus, together with the unit's Linux selector; with that change the unit's determinism theorem closes in under a minute. The change alters the constraints of two units and therefore the verifying keys, so it is staged with the next key-changing release rather than deployed alone.

10 Concrete efficiency

This section reports two metrics and one environment. The two metrics are the ones a user of the block pays: *emulator efficiency*, the rate at which the machine's software emulator turns a guest program into the record the prover consumes, together with the host memory and processor time that costs (§10.2); and *prover efficiency*, the rate at which that record becomes a proof, together with the trace area and proof size behind it (§10.3). The environment is deliberately narrow: one workload, one GPU model, one host, and interleaved repetitions on the same input, so that differences between configurations are attributable. Measurements taken on other projects' hardware are reported as context only (§10.6), and §10.8 records what the narrowness costs the conclusions.

10.1 Experimental setup

Workload. The workload is Ethereum mainnet block 25 907 955 executed by a MIPS32r2 build of the `reth` execution client with a witness trie represented as an arena. We use it because it is the workload on which this

class of system is publicly compared, and because its instruction mix is representative of the hash-intensive computations these machines are asked to prove: the guest spends its cycles hashing trie nodes, recomputing a Merkle root over state it does not hold, and verifying signatures. It is not, however, a general-purpose benchmark, and §10.8 records what that costs the conclusions. The guest is compiled with a word-wise `memcmp`, a zero-copy Keccak wrapper and an arena representation of the witness trie, which together bring the block to 288 million cycles from the 379 million a naive build costs; every prover measurement below uses that record, from a cached witness, so that all runs execute the identical instruction stream. The multi-GPU measurements of §10.5 use three further blocks of 420, 530 and 912 M cycles.

Hardware. One NVIDIA RTX 5090 (32 GB) in a host with one AMD EPYC 9355 processor and 925 GB of memory; the multi-GPU measurements use four such GPUs in the same host (driver 570.153.02, CUDA 12.8, Ubuntu 22.04). The prover is pinned to one GPU.

Metrics. *Wall-clock time* is the end-to-end time of the proving process from reading the cached input to writing the compressed proof, including host verification of every shard proof and of the compressed proof (2–3 s of the total, which a production service would omit). Two rates are reported, both in megahertz of guest execution. *Emulation rate* is guest cycles per second of the emulator's own wall-clock time; *proving throughput* is guest cycles per second of the whole proving process's wall-clock time. They are not comparable to each other—the second includes the first—and neither is a clock frequency. *Trace area* is the sum over units of rows × columns, in cells, from the emulator's shard-close census multiplied by the unit widths; Table 3 gives the widths after the changes of §10.4, while Table 5 reports the census at the widths that preceded them. *Proof size* is the byte length of the serialised compressed proof. GPU kernel time is attributed to kernel groups with `nsys` over a ten-shard sample.

Protocol. Each pair of configurations is measured in interleaved A/B/B/A order on the same GPU and the same input, and medians are reported; the spread over three repetitions was within ±1 s of the median for every row of Table 6 and within ±0.8 s for Table 7. The GPU under measurement was otherwise idle; a second, unrelated proving process ran on another GPU of the same host, so the host processor was not idle.

10.2 Emulator efficiency

The emulator is the block's front end: everything the prover consumes passes through it, and on a host driving several GPUs its cost, not the accelerator's, is what binds (below). Its two implementations differ by an order of magnitude in rate. On the block of §10.1 the just-in-time compiler emulates the guest at roughly 130 MHz, while the interpreter runs at 5–20 MHz depending on whether it is recording events; the two produce the same record (Section 3), so the compiler is a pure speed-up. The compiled emulator is therefore some twenty times faster than proving and the interpreter roughly comparable to it, so on a single GPU emulation is not the binding cost.

It becomes binding when one host feeds several GPUs, because each shard is re-emulated by the worker that proves it. A worker's replay and trace generation took approximately one second of host processor time per shard against half a second of GPU time, and the event stream written by one shard replay was approximately 1 GB. Table 4 lists the changes made under an explicit host budget; each is a byte-identical transformation of the record, so the proofs do not change, and each removes data that no constraint reads.

Two rules follow. An event should carry exactly what a column witnesses: three of the seven rows remove fields that only a debug assertion or an unused code path had kept alive; on the shared register records alone this is 44 bytes for every instruction event, one write record and two read records. And the process that coordinates several GPUs should never hold the whole execution: the coordinator's resident state is one shard's oracle and the guest image, so the block size is not bounded by host memory; only the shard size is bounded, by the accelerator's memory (§10.4). Both rules apply to any zkVM whose prover is fed by a re-executing host.

Remark 10.1 (Optimisations not reflected in the tables). Two changes measured after the tables were taken are not reflected in them: wider clock fences, which reduce the shard count by 7%, and heavier query grinding with device-side early exit, which reduces the proof by 7%. Both were measured at a fixed revision and shard size, and neither moves wall-clock time.

Table 4: Emulator and record pipeline under a host memory and processor budget. Sizes are per event or per shard of the block of §10.1; times are host-side, single process, three repetitions.

Change	before	after	effect
Per-cycle event: keep clock, `pc` pair and exit code only (no CPU table reads the rest)	184 B	20 B	events/shard 1000 → 462 MB; replay 700 → 435 ms
Memory-instruction event: drop the three fields no column reads	168 B	136 B	every load/store event
Register-access record: carry only the witnessed values (no shard fields; one arm for read/write)	48 / 24 B	20 / 16 B	every instruction event
Memory oracle shipped to a worker: pre-access value, stamp and shard as one byte run	24 B, 25 MB	12 B, 12 MB	per access, per shard; serialise 6–7 → 1 ms
Coordinator producer without checkpoints: no state clone or first-touch map per shard	22.9 s	20.4 s	per block; coordinator state O(shard)
Native (JIT) producer in the coordinator	12.3 s	6.1 s	per 420 M-cycle block
Flat guest memory: one 16-byte entry per word in a sparse mapping; untouched pages cost nothing; unconstrained blocks are copy-on-write views	paged table	flat, sparse	no per-block memory diff

Table 5: GPU kernel time of the core-proving stage by group (nsys, ten-shard sample) and trace area by unit (19.6 G cells over the whole block), deployed configuration before the changes of §10.4. Area is rows × columns at this row's shard count; the `Global` rows scale with the number of shards (§5.4).

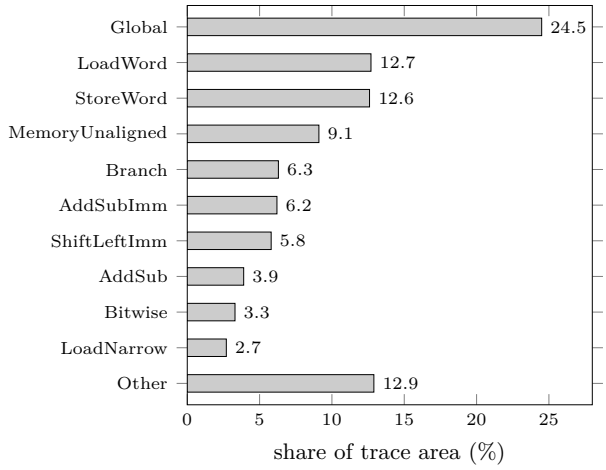
Kernel group	share	Unit	area share
LogUp-GKR layers (fold-and-sum, transitions, first layer)	42%	`Global` (48 M rows × 100)	24.5%
Zerocheck, compiled constraint kernels	16.5%	`LoadWord` (51 M × 49)	12.7%
Zerocheck, interpreted (wide precompile units)	7.1%	`StoreWord` (46 M × 53)	12.6%
Zerocheck fold	2.3%	`MemoryUnaligned` (30 M × 60)	9.1%
WHIR commit and open (Merkle absorb, DFT)	19%	`Branch` (22 M × 56)	6.3%
Jagged reduction	8%	`AddSubImm` (37 M × 33)	6.2%
Other	5%	`ShiftLeftImm` (20 M × 56)	5.8%

10.3 Prover efficiency: where the cost is

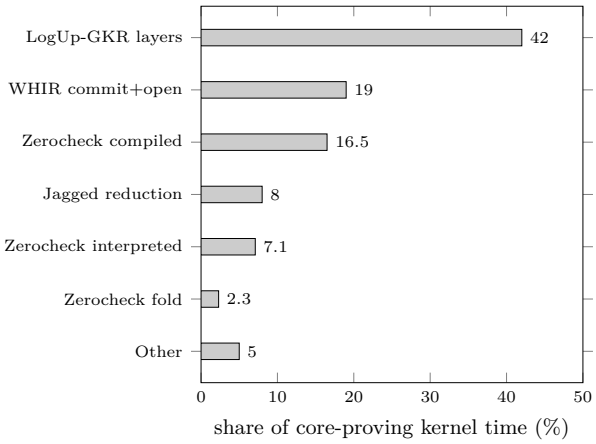
Table 5 pairs the kernel-time profile with the area census; Figure 8 plots them side by side. Kernel time is approximately 340 ms per shard against 500 ms of GPU time per shard, the difference being launch gaps and host synchronisation. Two observations follow. First, the lookup argument is the largest kernel consumer, and its cost scales with the number of *(row, interaction)* pairs—7.8 G on this block against 19.6 G trace cells—so that one interaction per row costs approximately as much as two columns; the memory-instruction units carry 29 interactions per row, 21 of them byte-table lookups, and account for 48% of all pairs. Second, the `Global` unit alone holds a quarter of the area at 2.5% of the pairs, because every word touched in a shard costs two of its rows. The cost model that follows is: area for the commitment and the zerocheck, and pairs for the lookup argument, with a pair weighted as two cells.

10.4 Prover efficiency: what reducing area buys

The cost model of §10.3 suggested the changes in Table 6. Replacing the 30-bit boolean sign decomposition, the eight offset bits and the seven chord-witness columns of the `Global` unit by three byte lookups removed 42 columns from 48 M rows and 3.8% of the wall-clock time; the compressed proof size is unchanged (617 618 bytes), while the core proofs shrink from 116 to 112 MB per block, which also shortens the leaf stage. Removing the redundant byte checks on memory-instruction accesses removed 5.7% of the LogUp-GKR pairs for 1.1% of the wall-clock time—less than the two-cells-per-pair rule predicts, because the lookup circuit's fold layers scale with padded heights rather than with pair count. An 11-column reduction of a 20 M-row unit is below the measurement noise; the byte-check row, at 1.0 s, sits at the edge of it. The largest single effect is the shard size: raising it by a quarter removes 16 of the block's shards—fewer than the cycle fence alone would suggest, because some shards close early on a unit's height budget—and saves 3.3 s, approximately 0.2 s per shard, which is the cost of one recursion leaf plus the per-shard setup. Larger shards were limited by



(a) Trace area by unit (19.6 G cells); `Global` and the three largest memory-instruction units hold 59%.



(b) GPU kernel time of the core-proving stage by group (nsys, ten-shard sample).

Figure 8: Where area and prover time go on the block, deployed configuration before the changes of §10.4.

Table 6: Single-GPU wall-clock time (block 25 907 955, core and compress stages, host-verified), medians of interleaved runs. Each row adds one change to the previous row.

Configuration	shards	wall-clock (s)	vs. previous
Deployed configuration, no lookup grinding	130	74.1	
+ `Global` 100 → 58 columns (§5.4)	130	71.2	−3.9%
+ memory-instruction accesses without byte checks (§5.3)	130	70.2	−1.4%
+ `ShiftLeftImm` 56 → 45 columns	130	70.4	neutral
+ shard size raised by a quarter	114	67.1	−4.7%
All four vs. deployed	114	67.1	−9.4%

GPU memory at this revision: at half again the baseline size the first-layer buffer of the LogUp-GKR circuit, 4.5 GiB, exceeds what the memory pool can place without repeated trim-and-retry and the prover runs 30% slower, and at twice the baseline the shard does not fit in 32 GB at all.

10.5 Prover efficiency: memory traffic, and scaling across devices

The profile identified the LogUp-GKR circuit as the largest kernel consumer (42%). A byte-level profile of its layers showed that the first layer paired adjacent fractions across the most-significant index bit, so that short units were carried through every layer of the tallest unit and the traffic summed over layers was 5.2× that of the first layer. Pairing on the least-significant bit instead lets each unit fold out after $\log_2$ of its own height. The change also frees enough memory for shards of twice the baseline size, which did not fit before it (§10.4). With both, the same block proves in 48.7 s on one GPU, against 67.8 s for the revision without either at the largest shard size it could run (−28% for the pair; the 67.1 s of Table 6 is a different revision), with the proof unchanged at 617 618 bytes.

Table 7 reports the scaling of that configuration from one to four GPUs on a 420 M-cycle block, against the deployed configuration of the first row of Table 6; the throughput columns give proved guest cycles per second of wall-clock time. Figure 9 plots throughput against GPU count for three block sizes (420, 530 and 912 M cycles; the larger blocks take 84.3/44.7/26.5 s and 124.3/65.0/38.0 s on one, two and four GPUs). Throughput is nearly independent of block size, within 20% across the three, and on the 420 M-cycle block scales by 1.9× from one GPU to two and by 3.1× to four; the two larger blocks reach 3.2× and 3.3× at four. The last row of Table 7 shows the change helping least at four GPUs: it shortens core proving, whose share of the wall-clock time falls as the recursion tail and the start-up ramp grow. The gap to linear scaling has

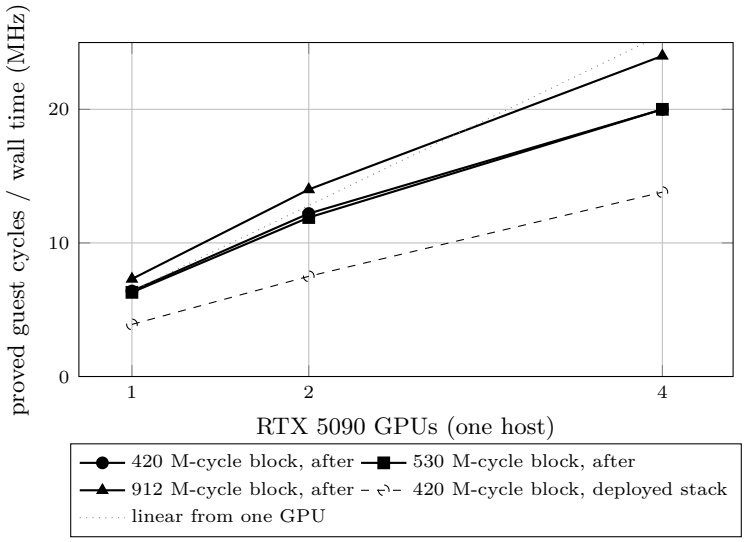


Figure 9: Proved-execution throughput against number of GPUs for three block sizes after the lookup-layer change, and the deployed configuration for comparison. The dotted line is linear scaling from one GPU.

Table 7: Wall-clock time per block (s, warm, three consecutive proofs of a 420 M-cycle block) and proved-execution throughput, one to four RTX 5090 GPUs in one host, before and after the lookup-layer change. Proof size 617 618 bytes in every case.

	wall-clock (s)			throughput (MHz)		
GPUs	1	2	4	1	2	4
Before (deployed configuration, baseline shards)	108.0	56.0	30.5	3.9	7.5	13.8
After (LSB pairing, doubled shards)	65.9	34.3	21.0	6.4	12.2	20.0
Speed-up	1.64×	1.63×	1.45×			

two components: the serial tail of the recursion tree after the last core shard, and the ramp at the start of a block, during which the emulator has not yet produced enough shards to keep every GPU busy—the same host-side cost as §10.2, seen from the accelerator's side.

10.6 Context: public block-proving provers

For context, the public block-proving dashboard [Eth26b] on 5 September 2026 showed 23-hour medians for single-GPU provers, on different hosts, fields, proof sizes and security regimes (Table 2), of 42 s (Airbender, RTX 5090), 73 s (ZisK, RTX 5090), 74 s (Ceno, RTX 4090), 92 s (cysic, RTX 4090) and 107 s with a 1.0 MB proof (Ziren, RTX 5090, before the changes reported here). On the blocks proved in common, Ziren took 2.8× Airbender's time, 1.6× ZisK's and 1.2× cysic's. These are not same-hardware comparisons, and no ranking should be inferred from them.

10.7 Answering the question of Section 1

The question this paper opened with had three parts, and the evidence for each is now in. *Is the prover competitive with the RISC-V systems?* Partly: on the public dashboard Ziren was the slowest of the five single-GPU provers listed, and the changes reported here reduce its time on our own block by a third, but the comparison is across different hardware, blocks and security regimes, and no same-hardware comparison against those systems exists. What can be said without qualification is that the machine proves a 288 million-cycle execution in under a minute on one consumer GPU, and scales to 20 MHz of proved execution on four. *Is the soundness stated under the weakest standard assumptions?* Within the proximity test, yes: the

query counts are set in the unique-decoding regime, with no list-decoding conjecture, and the components are composed by a union bound rather than reported individually. Outside it, two assumptions remain, the random oracle and the discrete-logarithm hardness behind the cross-shard digest. *Is the constraint system itself verified?* In part, and Section 9 is precise about which part: the emulator against an independent reference and a formal model, the units by extracted theorems of which a minority are closed, and the recursion machine not at all.

10.8 Threats to validity

Internal validity. All same-hardware comparisons are interleaved and repeated, and the measured spread is small relative to the reported effects; the one effect within the noise (the `ShiftLeftImm` change) is reported as neutral. The host processor was shared with an unrelated process during the single-GPU measurements, which can only have increased the reported times. *Construct validity.* Every figure in this section was taken with the hash configured as deployed, whose round schedule carried the count tabulated for a higher S-box degree than this field has (§9.3). Correcting it adds seven partial rounds to every permutation, and the permutation is the commitment hash, the transcript hash and the recursion machine's hash, so the correction moves these numbers in a direction we have not measured. Wall-clock time includes host verification, which a production deployment omits; throughput therefore slightly understates production throughput. Trace area is a model of prover cost, not a measurement of it; the kernel profile is the measurement, and the two are reported side by side. *External validity.* The workload is one Ethereum block, chosen because block proving is the public benchmark for this class of system; the instruction mix of other guests (Section 9 proves small conformance programs) differs, and the per-unit area shares would differ with it. The dashboard comparison of §10.6 spans different hardware, fields and security regimes and is reported as context only.

A Protocol details

A.1 The WHIR iteration as implemented

Section 7 states what one iteration does; this appendix gives it step by step. An *iteration* with folding factor k reduces membership in $\mathsf{CRS}[\mathbb{F}, L, m, \widehat{w}, \sigma]$ to membership of a new function f' in $\mathsf{CRS}[\mathbb{F}, L^{(2)}, m-k, \widehat{w}', \sigma']$: the number of variables drops by k while the domain only halves, so the code's rate falls from ρ to $2^{1-k}\rho$ (its redundancy $1/\rho$ grows) in the paper's convention; Ziren instead re-encodes the folded polynomial at an explicit per-round rate (below). An iteration proceeds as follows [ACFY24b, §2.1.3].

1. **Sumcheck rounds.** k rounds of sumcheck on $\sum_{\boldsymbol{b}} \widehat{w}(\widehat{f}(\boldsymbol{b}), \boldsymbol{b}) = \sigma$, producing challenges $\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_k)$ and reducing the claim to one about $\widehat{f}(\boldsymbol{\alpha}, \cdot)$.

2. **Folded oracle.** The prover sends $g : L^{(2)} \to \mathbb{F}$, honestly the evaluation of $\widehat{g} = \widehat{f}(\boldsymbol{\alpha}, \cdot)$, re-encoded at the round's rate.

3. **Out-of-domain samples.** The verifier draws $z_0 \in \mathbb{F}_{p^4}$ (two such points in Ziren); the prover answers $y_0 = \widehat{g}(\boldsymbol{z}_0)$ with $\boldsymbol{z}_0 = (z_0, z_0^2, \dots)$.

4. **Shift queries.** The verifier draws t indices $z_1, \dots, z_t \in L^{(2^k)}$, obtains $y_i = \mathsf{Fold}(f, \boldsymbol{\alpha})(z_i)$ by reading the 2^k symbols of f that fold to z_i (one Merkle leaf each), and draws a combination scalar γ. As with the out-of-domain point, $\boldsymbol{z}_i = (z_i, z_i^2, \dots)$ denotes the power vector of z_i.

5. **Recursive claim.** The new weight is $\widehat{w}'(Z, \boldsymbol{X}) = \widehat{w}(Z, \boldsymbol{\alpha}, \boldsymbol{X}) + Z \sum_i \gamma^{i+1} \mathsf{eq}(\boldsymbol{z}_i, \boldsymbol{X})$ and the new target $\sigma' = \sigma_k + \sum_{i=0}^{t} \gamma^{i+1} y_i$, where σ_k is the value the round's k sumcheck steps leave and the sum runs over the out-of-domain answer y_0 and the t shift answers; the protocol recurses on $(g, \widehat{w}', \sigma')$.

After the last iteration the prover sends the coefficients of the final polynomial in the clear and the verifier checks the residual constraint directly.

A.2 The jagged branching program and the assist sumcheck

Evaluating $\widetilde{f}_t$: the branching program. Proposition 3.2 of [HJR+25] writes

$$\widetilde{f}_t(\boldsymbol{z}_r, \boldsymbol{z}_c, i) = \sum_{y \in \{0,1\}^k} \mathsf{eq}(\boldsymbol{z}_c, y)\, \widehat{g}(\boldsymbol{z}_r, i, t_y, t_{y+1}),$$

where $g(a, b, c, d) = 1$ iff $b < d$ and $b = a + c$ as integers. The function g is computed by a width-four read-once branching program reading one bit of each of a, b, c, d per step (two bits of state: the carry of $a + c$ and whether the prefix of b is below the prefix of d), so by Holmgren–Rothblum [HR18] its multilinear extension is evaluated in $O(m)$ field operations by a dynamic program over the layers. Ziren implements exactly this program: four memory states, four bits read per layer (sixteen bit-states), evaluated in reverse over $m + 1$ layers. The sum over the 2^k columns is not done by the verifier directly—$K \approx 3.6 \cdot 10^4$—but delegated to the prover through the "jagged assist" of Section 5 of the paper: the prover proves

$$\mathsf{jagged\_eval} = \sum_{x,y \in \{0,1\}^{m+1}} \sum_k \mathsf{L}_k(\boldsymbol{z}_c)\, \mathsf{eq}\big((x, y), (\mathrm{bits}(t_k), \mathrm{bits}(t_{k+1}))\big)\, \mathsf{BP}(\boldsymbol{z}_r, \boldsymbol{z}^*, x, y)$$

by a $2(m + 1)$-variable sumcheck, where L_k is the k-th Lagrange weight of $\boldsymbol{z}_c$ and BP the branching-program polynomial; the verifier finishes with one evaluation of BP at the sumcheck point and an in-the-clear check of the closing identity against the public prefix sums.

B The core as delivered

This appendix collects the outward-facing description of the core: what an integrator connects, what may be varied, how it is embedded, and what is delivered with it. Section 4 explains why these are kept apart from the design itself.

B.1 Ports

Table 8 lists the interfaces of the core. Three of them are fixed by the statement of Section 1.1: a program image, an input stream and a hint stream go in, and a proof and its public values come out. The remaining interfaces are the ones an integrator connects: a verifying key derived once per program, a verifier in one of two forms, an extension interface for accelerators, and the key allowlist that binds a recursion proof to the advertised machine.

B.2 Configuration space

The core is parameterised, and the evaluation of Section 10 is a walk through part of its configuration space. The parameters are: the shard size, which trades the per-shard setup and the recursion leaf against the memory of the lookup circuit's first layer (Section 10 measures four values); the stacking height of the commitment, which sets the stripe count; the WHIR schedule—the number of committed rounds, their folding factors and code rates, the query counts, the out-of-domain samples and the query grinding—which sets the proof size and the soundness accounting (Section 7); the grinding on the lookup challenges; the recursion arity; and the accelerator set, which determines the unit set of the precompile shards and hence the verifying keys. The values of a given release are in its configuration file. Changing any parameter that enters a constraint system changes the verifying keys of the affected units and therefore the recursion-key allowlist, which is regenerated once per release; parameters that only enter the schedule (query counts, grinding) change the keys of the recursion programs but not of the core units.

B.3 Integration paths

The integrations below share the same ports. The common shape is a guest that hashes: it recomputes a Merkle root over data the verifier does not hold, checks an inclusion or update proof, verifies a chain of signatures or certificates, or attests a firmware image or a log. The guest is compiled by an ordinary MIPS

Table 8: Ports of the core. “Setup” interfaces are used once per program; “run” interfaces once per execution.

Port	When	Contents
Program image	setup	a little-endian MIPS32r2 ELF produced by the guest toolchain (GCC, LLVM or the Rust target); its text and initial data become the program ROM and the initial memory
Verifying key	setup	the Merkle commitment to the program ROM and the preprocessed tables (Section 5); 304 bytes, derived deterministically from the image; identifies the program in the statement
Input stream	run	the bytes the guest reads as its standard input; a witness
Hint stream	run	host-supplied data the guest reads through the hint syscall (for example the block witness); a witness, written into untouched memory and thereby bound by the memory argument
Public values	out	the digest of the committed output, the exit code, the entry and exit machine state, the address cursors of the global memory tables and the septic digest (Section 3)
Compressed proof	out	the root of the recursion tree, 603 KiB for the block of Section 10; optionally wrapped to a Groth16 proof over BN254 for on-chain verification (Section 8)
Verifier	deliverable	a verifier that checks a compressed proof on an ordinary host, small enough to embed in the consuming application; it carries the recursion-key allowlist
Accelerator interface	extension	a syscall number, an emulator hook that reads and writes guest memory at the invoking cycle, and a unit that receives the syscall tuple (arguments as 16-bit half-words plus the Linux flag) on the syscall bus and performs its memory accesses on the memory bus (Section 3)

toolchain, calls the hash accelerators of Section 3 where the volume justifies it, and the verifier checks one proof whose cost does not grow with the amount hashed.

Three specialisations are worth naming. In *block proving*, the deployed case, a coordinating process cuts a native execution into shards and ships a descriptor per shard to one worker process per GPU, which re-execute and prove their shards and the recursion nodes above them; §10.2 describes the host budget this pipeline was engineered against. In *hybrid fault and validity proving*, one program image is adjudicated by a fault proof and certified by a validity proof without a second toolchain, because the core executes the instruction set Cannon and Kona already execute [Opt23, OP 24]. In *embedded and IoT workloads*, a program compiled by a device’s own MIPS toolchain is executed by a remote prover and a gateway checks the proof, so the statement is the execution of the device’s software rather than a property of the device (§1.5). We report no measurement of the wrapped on-chain path, and Section 9.3 records two omissions found in the terminal circuits it depends on.

B.4 Verification collateral

The block is delivered with the collateral of Section 9: the specification-derived conformance suite and the Cannon suite with their reference results; the executable Lean 4 model of the ISA and its two checks against the emulator; one extracted determinism theorem per unit and opcode selector, with the tactic that discharges them and the list of open ones; the soundness configuration and the generated accounting report for the deployed schedule; and the defects register (Table 9). An integrator therefore verifies once that the toolchain produces the image it expects and that the verifier binary and the key allowlist are the delivered ones. The correspondence between the constraint system and the ISA, and the uniqueness of the constraint system’s behaviour, are addressed by the collateral to the extent Section 9 records—the first by testing against an independent reference and a formal model, the second by 17 of 106 closed unit theorems and a uniqueness theorem that is stated rather than machine-checked—and do not have to be re-established per integration.

Table 9: Defects surfaced by the verification flow. "Theorem" means the extracted determinism theorem of the unit; "proving" means an end-to-end proof of a real block or of the test programs; "review" means a line-by-line comparison of this design against the RISC-V system it descends from, reading both implementations side by side.

Component	Defect	Found by	Resolution
Syscall unit	argument half-words not determined by the reduced argument ($2^{32} > p$)	theorem	half-words carried on the syscall bus; theorem closes
Syscall unit	result columns gated by a selector that no input pins	theorem	selector carried on the syscall bus; theorem closes
Extractor	shard and clock fields of syscall interactions dropped, leaving outputs without inputs	theorem	extractor fixed
Shape configuration	range table absent from the preprocessed heights; every shape-fixed proving test failed	proving	height registered
Standalone verifier	recursion-key Merkle root computed from the bare key list while the prover pads to the tree height	proving	verifier pads the leaves
Proving service	configured to skip the in-circuit key check, producing proofs the standalone verifier rejects	proving	service configuration
Recursion, terminal stages	the completeness flag gates every completeness predicate—including the one forcing the cross-shard sum to zero—and neither the shrink nor the wrap circuit pinned it, so a proof of an execution prefix could be wrapped	review	both terminal circuits assert it; the deferred program can no longer emit it set
Wrap circuit	the recursion-key allowlist root was not a public input, so no external verifier could require the published one	review	exposed as a public input and checked by the standalone and on-chain verifiers
Hash parameters	partial-round count taken from the tabulation for S-box degree seven rather than three, giving less algebraic-attack margin than intended	review	schedule corrected; the measurements of Section 10 predate the change

C Defects surfaced by the verification flow

References

[ACFY24a] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. STIR: Reed–Solomon Proximity Testing with Fewer Queries. Cryptology ePrint Archive, Paper 2024/390, 2024.

[ACFY24b] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification. Cryptology ePrint Archive, Paper 2024/1586, 2024.

[AHIV17] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Ligero: Lightweight sublinear arguments without a trusted setup. In *ACM CCS*, 2017.

[AST24] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: SNARKs for virtual machines via lookups. EUROCRYPT 2024; Cryptology ePrint Archive, Paper 2023/1217, 2024.

[Axi24] Axiom. OpenVM: A performant and modular zkVM framework. https://openvm.dev/whitepaper.pdf, 2024.

[Bar89] David A. Barrington. Bounded-Width Polynomial-Size Branching Programs Recognize Exactly Those Languages in NC^1. In *Journal of Computer and System Sciences*, volume 38, pages 150–164, 1989.

[BEG+94] Manuel Blum, Will Evans, Peter Gemmell, Sampath Kannan, and Moni Naor. Checking the Correctness of Memories. In *Algorithmica*, volume 12, pages 225–244, 1994.

[BSBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed–Solomon Interactive Oracle Proofs of Proximity. In *Proceedings of the 45th International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 14:1–14:17, 2018.

[BSCI+20] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for Reed–Solomon codes. In *FOCS*, 2020.

[BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive Oracle Proofs. In *Theory of Cryptography — TCC 2016-B*, pages 31–60, 2016.

[BSGKS20] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. DEEP-FRI: Sampling outside the box improves soundness. In *ITCS*, 2020.

[CJSC23] Zhigang Chen, Yuting Jiang, Xinxia Song, and Liqun Chen. A survey on zero-knowledge authentication for Internet of Things. *Electronics*, 12(5):1145, 2023.

[Con24] Consensys. gnark: A fast zk-SNARK library in Go. https://github.com/Consensys/gnark, 2024.

[CY24] Alessandro Chiesa and Eylon Yogev. Building Cryptographic Proofs from Hash Functions. https://github.com/hash-based-snargs-book, 2024. Textbook treatment of the BCS transformation, round-by-round soundness and state restoration.

[Cys25] Cysic. Cysic: hardware-accelerated proof generation. https://cysic.xyz, 2025. Listed on the public block-proving dashboard [Eth26b].

[DDGM23] Heini Bergsson Debes, Edlira Dushku, Thanassis Giannetsos, and Ali Marandi. ZEKRA: Zero-knowledge control-flow attestation. In *ACM ASIA Conference on Computer and Communications Security (ASIA CCS)*, 2023.

[DP23] Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. Cryptology ePrint Archive, Paper 2023/1784, 2023.

[DP24] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinears over binary towers. Cryptology ePrint Archive, Paper 2024/504, 2024.

[EFG22] Liam Eagen, Dario Fiore, and Ariel Gabizon. cq: Cached quotients for fast lookups. Cryptology ePrint Archive, Paper 2022/1763, 2022.

[EH24] Shahriar Ebrahimi and Parisa Hassanizadeh. From interaction to independence: zkSNARKs for transparent and non-interactive remote attestation. In *Network and Distributed System Security Symposium (NDSS)*, 2024. IACR ePrint 2024/1068.

[Eth26a] Ethereum Foundation. soundcalc: security calculator for hash-based proof systems. https://github.com/ethereum/soundcalc, 2026.

[Eth26b] Ethproofs. Ethproofs: real-time proving of Ethereum blocks. https://ethproofs.org, 2026.

[FS87] Amos Fiat and Adi Shamir. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology — CRYPTO '86*, pages 186–194, 1987.

[GKR08] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. Delegating Computation: Interactive Proofs for Muggles. In *Proceedings of the 40th ACM Symposium on Theory of Computing (STOC)*, pages 113–122, 2008.

[GKS23] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A faster version of the Poseidon hash function. In *AFRICACRYPT*, 2023.

[GLS+21] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad Wahby. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. Cryptology ePrint Archive, Paper 2021/1043, 2021.

[GW20] Ariel Gabizon and Zachary J. Williamson. plookup: A simplified polynomial protocol for lookup tables. 2020.

[Hab22] Ulrich Habök. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Paper 2022/1530, 2022.

[Har25] David Harold. MIPS at 40. Jon Peddie Research, https://www.jonpeddie.com/news/mips-at-40/, January 2025.

[HJR+25] Tamir Hemo, Kevin Jue, Eugene Rabinovich, Gyumin Roh, and Ron D. Rothblum. Jagged Polynomial Commitments (or: How to Stack Multilinears). Cryptology ePrint Archive, Paper 2025/917, 2025.

[HLP24] Ulrich Haböck, David Levit, and Shahar Papini. Circle STARKs. Cryptology ePrint Archive, Paper 2024/278, 2024.

[HR18] Justin Holmgren and Ron Rothblum. Delegating Computations with (almost) Minimal Time and Space Overhead. Electronic Colloquium on Computational Complexity, Report TR18-161, 2018.

[LFKN92] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic Methods for Interactive Proof Systems. In *Journal of the ACM*, volume 39, pages 859–868, 1992.

[Lit24] Lita Foundation. Valida ISA specification, version 1.0: A zk-optimized instruction set architecture. https://github.com/lita-xyz/valida-releases, 2024.

[LZZ+24] Tianyi Liu, Zhenfei Zhang, Yuncong Zhang, Wenqing Hu, and Ye Zhang. Ceno: Non-uniform, segment and parallel zero-knowledge virtual machine. Cryptology ePrint Archive, Paper 2024/387, 2024.

[Mat25] Matter Labs. ZKsync Airbender: A RISC-V zkVM over Mersenne-31. https://github.com/matter-labs/zksync-airbender, 2025.

[Med26] MediaTek Inc. MT7621A/N dual-core network processor. https://www.mediatek.com/products/home-networking/mt7621, 2026. 880 MHz MIPS 1004KEc dual-core Wi-Fi/router SoC. Accessed September 2026.

[Mic26] Microchip Technology Inc. MIPS32-core-based microcontrollers (PIC32M). https://www.microchip.com/en-us/products/microcontrollers/32-bit-mcus/pic32m, 2026. Accessed September 2026.

[MIP26] MIPS Tech LLC. IP processors. https://mips.com/products/hardware/, 2026. States "over 8.5B MIPS-based devices shipped". Accessed September 2026.

[Nex24] Nexus Labs. Nexus 1.0: Enabling verifiable computation. https://whitepaper.nexus.xyz/, 2024.

[OP 24] OP Labs. Kona: A rust implementation of the OP Stack fault-proof program. https://github.com/op-rs/kona, 2024.

[Ope26] OpenWrt Project. Targets and subtargets (`ath79`, `ramips`). https://openwrt.org/docs/techref/targets/start, 2026. Accessed September 2026.

[Opt23] Optimism Foundation. Cannon: A MIPS interpreter for Optimism fault proofs. https://github.com/ethereum-optimism/optimism, 2023.

[PCW+23] Shankara Pailoor, Yanju Chen, Franklyn Wang, Clara Rodríguez, Jacob Van Geffen, Jason Morton, Michael Chu, Brian Gu, Yu Feng, and Işil Dillig. Automated detection of under-constrained circuits in zero-knowledge proofs. In *PLDI*, 2023.

[PH23] Shahar Papini and Ulrich Habök. Improving logarithmic derivative lookups using GKR. Cryptology ePrint Archive, Paper 2023/1284, 2023.

[Pol24] Polygon Zero and contributors. Plonky3: A toolkit for STARKs over small fields. https://github.com/Plonky3/Plonky3, 2024.

[Pol25] Polygon Labs. ZisK: A high-performance zkVM. https://github.com/0xPolygonHermez/zisk, 2025.

[QV15] Nguyen Anh Quynh and Dang Hoang Vu. Unicorn: The ultimate CPU emulator. Black Hat USA; https://www.unicorn-engine.org/, 2015.

[RIS24a] RISC Zero. STARK soundness calculator. https://github.com/risc0/risc0/tree/main/risc0/zkp/src/prove/soundness.rs, 2024.

[RIS24b] RISC Zero, Inc. RISC0: A general-purpose zero-knowledge virtual machine. https://www.risczero.com, 2024.

[SD21] Xavier Salleras and Vanesa Daza. ZPiE: Zero-knowledge proofs in embedded systems. *Mathematics*, 9(20):2569, 2021. IACR ePrint 2021/1382.

[Sta21] StarkWare Industries. Cairo: A Turing-complete STARK-friendly CPU architecture. https://www.cairo-lang.org, 2021.

[Sta23] StarkWare. ethSTARK documentation—version 1.2. Cryptology ePrint Archive, Paper 2021/582, 2023.

[STW24] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with Lasso. EUROCRYPT 2024; Cryptology ePrint Archive, Paper 2023/1216, 2024.

[Suc24] Succinct Labs. SP1: A performant, open-source, contributor-friendly zkVM. https://github.com/succinctlabs/sp1, 2024. Accessed 2026.

[Suc25a] Succinct Labs. SP1 Hypercube: Real-time proving of Ethereum blocks. https://blog.succinct.xyz/sp1-hypercube/, 2025. Technical report and open-source implementation.

[Suc25b] Succinct Labs. sp1-jit: Just-in-time RISC-V executor for SP1. https://github.com/succinctlabs/sp1, 2025. Component of the SP1 repository.

[The25] The ZKM Project. Ziren: A zero-knowledge MIPS virtual machine. https://docs.zkm.io, 2025.

[Tur21] Jim Turley. Wait, what? MIPS becomes RISC-V. *EE Journal*, March 2021. https://www.eejournal.com/article/wait-what-mips-becomes-risc-v/.

[ZCF23] Hadas Zeilberger, Binyi Chen, and Ben Fisch. BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes. Cryptology ePrint Archive, Paper 2023/1705, 2023.

[ZKM24] ZKM. zkMIPS: A high-performance zero-knowledge proof system for the MIPS architecture. https://github.com/zkMIPS/zkMIPS, 2024.